Learning, Search and Evidence in a Pokémon Showdown VGC Agent

From Policy Learning to Simulator Search and Live Ladder Experiments: Fourteen Campaigns, Software Reliability, and a Measured Team Breakthrough

`pokemon-showdown-player`
Technical report grounded in the repository implementation

31 October 2026 (third edition)
Implementation snapshot: `bee08dc`; simulator pin `c046106`
Evidence cutoff: 31 October 2026, 16:28 UTC

Abstract

This report follows a complete autonomous player for the Champions VGC 2026 Regulation M-C doubles ladder from an action-conditioned PPO actor to determinized simultaneous-move search in the official simulator. A local Apple M5 runs the search, MPS model fitting, campaign evaluation, recording and a Playwright transport; no language model chooses live turns. Learned opponent and preview models use public replays, while private requests and search traces support reproducible diagnosis. We retain the actor–critic, GAE/PPO derivations and earlier evidence as an explicitly dated development history, then extend the account through campaigns C1–C14.

The central result is the difference between better predictions, better surrogate-game solutions and stronger ladder play. Sixty-one value candidates, additional search compute, improved opponent predictions, scouting, a live factorial experiment and a shared Bayesian root solver did not establish a further live gain. C13 instead changed the team while holding the search and preview fixed. Its randomized ladder comparison accepted the challenger after 897 games: 249/448 wins versus 210/449, an estimated +61.9 Elo on the implemented score-difference scale, with a normal-approximation sequential log-likelihood ratio of 3.054 above the 2.890 boundary. The subsequent 455-game champion cohort has performance rating 1344 ± 33 (descriptive approximate 95% interval), compared with the earlier rain plateau near 1200–1300. This is evidence for the team–pilot combination, not a learned turn-policy breakthrough. C14 tests a different preview on that team and is unresolved at the cutoff. The report specifies stopping rules, a documented C5 promotion amendment, reconstruction and transport repairs, hardware measurements, and the remaining statistical and information-set limitations.

Keywords: Pokémon Showdown, VGC doubles, determinization, regret matching, PPO, action masking, Apple MPS, paired sign test, Elo.

Contents

1 Introduction and scope 5

2 System overview 6

3 The battle as a partially observed stochastic game 9

4 Candidate actions and doubles constraints 11

5 Features, profiles and the strategic prior 13

6 The action-conditioned actor–critic 16

7 The data recorded for every match 19

8 Learning from outcomes: policy gradients, GAE and PPO 21

9 The training pipeline on CPU and Apple MPS 26

10 Promotion: a paired test and what it can certify 29

11 Decision-time search 32

12 One live turn: browser transport, guards and viewer 37

13 Opponent-move scouting and team selection 40

14 Historical evidence: the actor and first search deployment 42

15 Fourteen campaigns: from prediction to playing strength 46

16 Live experimentation and the measured breakthrough 51

17 Software and compute: making experiments survive real play 54

18 Limitations and research directions 57

A Notation 58

B Corrections to the first edition 59

C Status of second-edition defects and subsequent repairs 60

D Changes from the second edition 61

E Source-to-claim map 61

F Reproduction and build 62

List of Figures

1 Responsibilities at the 31 October cutoff. Orange boxes run simulator search. Preview is a separately chosen model, with C14's assignment fixed for the game. The learning column is campaign-driven; it does not imply continuous PPO updates or an active strength gate. The dashed deployment path adopts tested settings between games. MCP remains an optional supervised path. 7

2 Chronology of the public commits from `153e3db` to `0c00ec2` (7–9 October 2026). The results are recomputed in Section 14. 8

3 The deployed actor–critic. The final actor layer is initialized to zero, so a fresh actor equals $\mathrm{softmax}(q_0/\tau)$. The optional `matchup-v1` variant adds a bilinear term $u^\top(e_a \odot Me_s)$ to the logit. 17

4 The recorder-3 episode. Learning uses the orange arrays; auditing and replay use the snapshot. For the last 20 ladder games the median document was 1.30 MB (range 0.73–2.06 MB). In a 7-step game, feature arrays took 750 KB and snapshots 440 KB of 1.20 MB. 20

5 Data lifecycle and the storage budget. Above 90% of the budget, collection, learning and evaluation defer. Above 85%, finished candidates' input snapshots are compacted down to 75%. Collecting checkpoints are kept once per digest, so every recorded likelihood can be recomputed. 21

6 GAE on a four-decision episode with a terminal win. Only the last residual contains the reward. Each earlier advantage adds its own residual to the next advantage, discounted by $\gamma\lambda$. 23

7 The lifecycle of experience. Orange boxes are gates that can stop the flow. Steps 1 and 7 run on the CPU, step 5 on the MPS GPU. Promotion (step 10) is called by the player between games, never by a worker. Episodes are marked consumed when the candidate is saved, before it is evaluated, so "trained" means *used for a candidate*, not *deployed*. 26

8 Schematic timeline of the lanes; durations are not to scale. The collector and evaluator share one pool of simulator slots. Within the default budget of four, the evaluator gets half (calibrated locally to three) and the collector the rest. The learner owns the GPU. The live runner is outside the pipeline and only ever consumes a staged candidate at a game boundary. 27

9 The paired design on twelve illustrative seeds. Only the five discordant seeds bear on the comparison. Here $g = 4$, $\ell = 1$ and $p = \mathbb{P}(\mathrm{Bin}(5, \frac{1}{2}) \geq 4) = 0.1875$, so this run would fail. 30

10 The search agent. Each world fixes complete hypothetical opposing sets and copies all public state. Screening keeps at most 24 of our joint actions and 20 of the opponent's. The payoff matrix of a world therefore needs at most 480 one-turn simulations, plus up to $4|\mathcal{B}| + 4|\mathcal{A}|$ for screening. Six initial worlds may be extended by six on close decisions within the adaptive time budget. The C5 leaf and selective continuation modify the basic one-turn game. Team preview uses a separate learned model. . . . 32

11 One live turn under search. Step 8 is the freshness check after the decision. Step 9 installs a capture-phase guard that cancels any click once the server's rqid has changed. Preview turns replace steps 4–6 with the selected pvalue or imitation preview model (originally the greedy actor). In recorded mode, steps 4–6 are `Brain.decide`. 38

12 Paired evaluations. Filled markers passed their gate and hollow markers failed. The orange diamond is the search agent, compared with the learned policy on the same 45 seeds. Bars are approximate 95% intervals for a paired difference, $\pm 1.96\sqrt{[(g+\ell)/n - ((g-\ell)/n)^2]/n}$. The `opening-v1` panel did not report its discordant counts. * Met $p \leq 0.05$ but missed the then-required 10-point minimum by one win. † One pass among 129 gates; see Proposition 10.2. 43

13 Live rating under the search agent on 9 October 2026. Gaps are games whose terminal log lacked our rating line. The climb from 1182 to 1464 took place in the symmetric-K region, where Proposition 14.1 predicts no drift for a player of stable strength. 45

14 Chronological performance cohorts, with sample size in parentheses. The Mev cohort joins all completed C13final assignments across the permanent team-setting restart. The earlier rows are contiguous non-A/B policy segments 6, 7, 8 and 13 in the frozen monitor snapshot. Time, opponents, repairs and policy differ across rows: this chart is not a causal decomposition. 52

1 Introduction and scope

Can one laptop play a current VGC format without a human choosing its turns, learn from the resulting experience, and measure whether a proposed change actually helps? The repository implements the full chain: connection and legal choices, an actor–critic and its training records, official-simulator search, public-replay models, a browser transport, and persistent experiments on both local opponents and the live ladder. Its research history is also a history of changing what counted as evidence.

The first edition (7 October) described MCP-supervised actor play. The second (9 October) documented the move to browser-driven search, its promising 45-game paired comparison and its first 41 live games. The third edition extends the record through 31 October and fourteen numbered campaigns. It reports a long plateau, unsuccessful attempts to improve the search and learned models, and a successful live team comparison. Crossing 1200 once was already possible in the early trajectory; the new result is a controlled improvement over the established rain pilot and a sustained post-promotion performance estimate above that level. A transient peak and a repeatable gain are different claims.

1.1 The system at the cutoff

Live turns use the C5 search configuration: six sampled hidden worlds, common random numbers, up to six additional worlds on close decisions, a win-probability-shaped leaf, selective Protect continuation and speed inference. The baseline preview is the opponent-aware C4 matchup-value model `pvalue-661448b6a320fce0`. C13 selected `SunRain-Mev-Brisbane` over the long-used Recife rain team. C14 now randomizes the preview between that baseline and a human-imitation argmax model, with team and turn search held fixed. The old PPO machinery remains implemented but its service is paused; it is not improving the live turn policy after every game.

1.2 Historical scope and kinds of evidence

The implementation baseline for this edition is public commit `bee08dc` (full hash in the evidence snapshot). The new live audit is frozen at 16:28:15 UTC on 31 October 2026. It contains 7,734 completed, rated search games, deduplicated by battle room, with 3,864 wins. This aggregate mixes policies and experiments and is not an estimate of the current champion's strength. Public aggregates, source hashes and numerical verification are provided in `evidence-20261031.json` and `collect_evidence.py` beside the paper. Raw requests, account identities and battle logs remain private. No battles, training, live deployments or campaign changes were performed for this article update.

Three kinds of statement remain separate throughout:

1. **Implemented contracts**, traced to named source files.
2. **Conditional mathematics**, proved under explicit assumptions; a certificate for a surrogate game is not a certificate for full VGC.
3. **Measurements**, with their date, population, stopping rule and limits. Chronological deployment segments are descriptive; randomized ladder arms supply the stronger treatment comparison.

Sections 5–10 preserve the actor-era architecture and proofs as the 7–9 October baseline, with scope notes rather than silently reinterpreting old measurements as current ones. Section 14 likewise

preserves the early evidence. New campaigns, evaluation methods and software advances are in Sections 15–17. Appendix C updates the status of defects identified by the second edition.

1.3 Format and pinned mechanics

The target is `gen9championsvgc2026regmc`: doubles, bring six and pick four, open team sheets when exchanged, one Mega Evolution per battle, no Terastallization, Item and Species Clauses, and Champions stat points instead of conventional EVs and IVs [30]. The simulator dependency remains pinned to official Showdown commit `c046106cbe075931b1ff8d8b800ff5be47a85f96` [20]. The `gen9` prefix alone does not specify these mechanics. Format, simulator revision, exact team, model assets and effective configuration are part of a reproducible experiment.

Research question	Evidence and qualification
Search rather than actor turns?	Early paired comparison: search 30/45, actor 19/45; 14 gains, 3 losses, exact one-sided $p = 0.0064$.
Can learned predictions improve search?	Value, opponent and preview models often improved predictive metrics; the later campaigns did not establish a corresponding live turn-policy gain.
Does more compute help?	C6's larger world and screening budgets failed their gates; the experiments do not prove that sampling noise is absent.
Did game-theoretic root solving help?	C12: 46 gains, 59 losses at 400 pairs; futility rejection despite small sampled-game solver gaps.
What broke the plateau?	C13 team change: 897 randomized games, +61.9 Elo estimate, sequential H1; subsequent champion performance 1344±33 over 455 completed games.
What is still being tested?	C14 preview A/B: 9 completed games at the cutoff, below the 300-game minimum; no decision.

Table 1: Third-edition headline evidence. Offline gains/losses count discordant pairs, whereas live A/B counts individual ladder games. Performance intervals are descriptive, not sequential confidence sequences.

2 System overview

2.1 Components and responsibilities

Figure 1 shows the current division of responsibilities. It has three columns. The *play* column talks to Showdown. The *decide* column chooses actions. The *learn* column turns recorded games into candidate models and decides whether to deploy them. The arrows that cross columns are narrow on purpose. A decision procedure receives a context built from public information and the player's own request, and returns one complete choice string. A learner receives completed, validated records and returns a checkpoint or a fixed set of options. Model pointers and experiment settings change at matchmaking boundaries. Source deployment selects a tested, pinned worktree and gracefully restarts the runner. Offline gates use separately pinned source; current development edits do not alter running experiments.

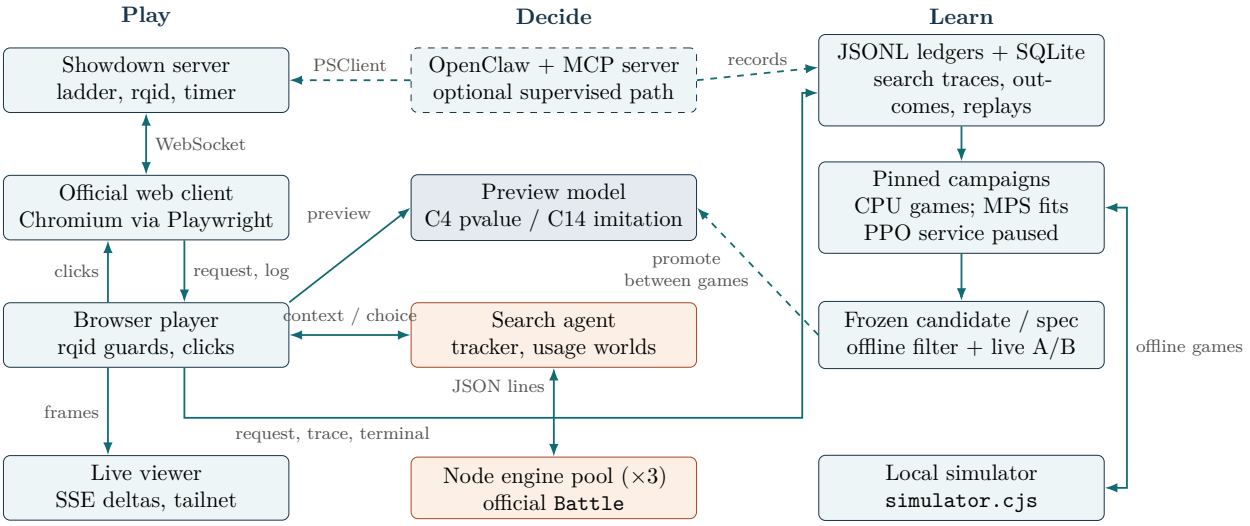


Figure 1: Responsibilities at the 31 October cutoff. Orange boxes run simulator search. Preview is a separately chosen model, with C14's assignment fixed for the game. The learning column is campaign-driven; it does not imply continuous PPO updates or an active strength gate. The dashed deployment path adopts tested settings between games. MCP remains an optional supervised path.

Process	Device	Role and cadence
Browser runner	CPU	One account lock. Plays ladder games through the official client, with a 30-second gap between games. Adopts experiment settings before each matchmaking search.
Search engines	CPU (3 Node)	Long-lived simulator processes. Each turn they receive 2 of the 6 initial worlds; close decisions may add 6. Later live medians are about 5–6 s.
Campaign controller	CPU	Pinned, restartable controller; balances arms, records first stopping decisions, and refuses to replace another active test.
Offline workers	CPU	Protocol-specific simulator games, frozen assets, same-seed retries; registered groups pause during live search.
Model fitting	MPS GPU	Value, screening, human-policy and preview models. Dedicated campaigns use chronological tests and fixed protocols; PPO is paused.
Live viewer	CPU	Publishes filtered public frames every 0.2 s over SSE.
Cold archiver	CPU	Every 900 s, gzip-archives completed inference inputs with SHA-256 verification.

Table 2: Long-running processes on the always-on Apple M5 host (10 cores, 16 GB unified memory). The historical lanes are in Section 9; later campaign operations are in Section 17.

2.2 Processes, devices and cadence

2.3 Project timeline

Figure 2 places the changes analysed in this report in time. The dates explain an important feature of the evidence. The 847-game live record spans three actor generations and two transports, so it is not a measurement of any single policy.

7 Oct • First edition; MCP harness; 100-game campaign (33–67)

7 Oct • Paired sign-test gate; turn temperature 0.25 promoted (95→129 of 200, $p \approx 10^{-6}$)

7 Oct • Recorder snapshots; feature profiles; colored-HP fix (new profiles only)

8 Oct • Integrity pass: likelihood check, recorder 3, 64-bit seeds, `ladder-v1` curriculum

8 Oct • Three-lane CPU/MPS pipeline; source-generation reloads

8 Oct • 375 games (146–229); opening, Mega and matchup candidates rejected

8 Oct • `strategic-v1` prior promoted (final 125 vs. 106 of 160)

9 Oct • `strategic-v2..v4` rejected; v4 fails a fresh confirmation

9 Oct • Old campaign stopped at 847 games (348–499); browser transport

9 Oct • Search agent deployed (offline 30 vs. 19 of 45)

9 Oct • PPO update passes the 60-game gate (38→46); 150 GB storage budget

9 Oct • Offline PPO matrix: no arm significant after Holm correction

Figure 2: Chronology of the public commits from `153e3db` to `0c00ec2` (7–9 October 2026). The results are recomputed in Section 14.

2.4 The later research timeline

The original timeline above ends at the second edition. The next three weeks changed both the decision system and the standard of evidence:

Date (October 2026)	Development milestone
9–13	Value-learning loop, common random numbers, adaptive worlds, learned-screening data, C1–C3 failures.
14–16	C4 opponent-aware preview; C5 risk/horizon/speed bundle and room-cache repair; pinned live deployment.
17–19	Mechanics fixes; C6–C8 fail; C9 begins randomized live factorial experimentation instead of relying only on imitation gates.
22–26	Pooled gates and persistent C10; M5 measurements; Illusion/Recharge repairs; C11 residual rejected; C12 shared-root candidate then rejected.
26–30	C13 compares teams on the ladder; browser team-cache/watchdog fixes; Mev wins round two at 897 games.
31	Mev becomes the runner default; C14 begins a team-specific preview A/B.

The protocol and outcome for each campaign appear in Section 15. Correctness fixes and service changes are recorded independently of claims about strategic improvement.

2.5 Two transports, one decision contract

The repository keeps two ways to reach Showdown. Both obey the same contract: a decision is one complete choice string for one actionable request, and it is taken from the request-visible candidate list $\mathcal{A}(r)$ of Section 4.

- **MCP harness.** `ps_mcp_server.py` registers 41 tools. The groups are connection, teams, play, model, learning and research. A single `PSClient` WebSocket logs in with a challenge-string assertion and sends `ROOMID|/choose CHOICE` [14, 15, 19]. OpenClaw's native registry exposes the tools as `pokemonshowdown__ps_*`. A retained session keeps the server process across turns; a detached one-shot run retires it [16]. This path still records decisions and can resume one pending episode after a disconnect.
- **Browser runner.** `scripts/browser_play.py` drives the official client with Playwright. It reads `app.rooms[id]` and clicks visible controls. Decisions come from local inference or search; no language model is in the turn loop (Section 12).

The two paths have different recording rules. MCP play can record sampled PPO data. A browser game played with `-search` is never a PPO sample, because its actions do not come from the actor's distribution. It retains a private decision ledger, produces public-log labels, and can supply reconstructed value positions and screening targets (Section 17).

Implementation: `ps_mcp_server.py`, `ps_client.py`, `harness.py`, `ml/live.py`, `ml/browser.py`, `scripts/browser_play.py`, `scripts/browser_service.py`, `scripts/learning_service.py`.

3 The battle as a partially observed stochastic game

3.1 State, actions and observations

Let s_t be the complete battle state at a decision boundary. It includes both teams' hidden sets, current HP, statuses, field conditions, pending mechanics and the simulator's random-number state. Player 1 chooses a complete action a_t and player 2 chooses b_t simultaneously. Chance ξ_t affects the transition:

$$s_{t+1} \sim P(\cdot \mid s_t, a_t, b_t), \qquad o^i_{t+1} \sim O_i(\cdot \mid s_{t+1}), \quad i \in \{1, 2\}. \tag{1}$$

A real trace also contains team preview, forced replacements and waiting requests. In the learner, the index t counts *recorded actionable requests*, not battle turns. Holding the opponent's strategy fixed reduces the game to a single-agent partially observed control process. If the opponent's behaviour depends on its own history, that history must be part of the latent state.

Write h_t for the information available to our player before decision t: the public protocol prefix, our own private requests and our past actions. An exact Bayesian belief is $\beta_t(s) = \mathbb{P}(s_t = s \mid h_t)$. For a fixed opponent with joint kernel $\bar{P}$ and observation kernel O, it updates by

$$\beta_{t+1}(s') = \frac{O(o_{t+1} \mid s') \sum_s \bar{P}(s' \mid s, a_t) \beta_t(s)}{\sum_u O(o_{t+1} \mid u) \sum_s \bar{P}(u \mid s, a_t) \beta_t(s)}, \tag{2}$$

whenever the denominator is positive [9, 27]. Nothing in the repository computes Equation (2). The neural policy compresses h_t into a feature vector $x_t = \phi(h_t)$ (Section 5). The search agent replaces the belief with a *sampling distribution* over opposing sets, built from ladder usage statistics and

conditioned on what has been revealed (Section 11). Both are approximations to β_t, and neither is calibrated.

3.2 What the format reveals

Champions M-C makes some hidden information public. Team preview reveals all six opposing species. When open team sheets are exchanged, the opposing species, items, abilities, moves and natures become known. Stat-point allocations stay hidden, and IVs are unused in this format. The audit in `docs/match-data-collection.md` found this pattern directly. Over 50 audited games, our own nature and stat points were stored in all 50. The opponent's nature was disclosed in 2, and its stat points in none. Open sheets are therefore informative but not complete. Only the damage-relevant stat spread remains latent once sheets are shown, but that spread alone moves damage rolls across KO thresholds.

Interpretation. An opposing Pokémon at 50% HP is an observation. Its unrevealed item is a latent variable. A predicted Protect is a learned forecast. A sampled stat spread is one hypothesis drawn from a population model. None of these is a recovered hidden state.

3.3 Non-anticipation

Both decision procedures read only the request and the public prefix. This can be stated precisely.

Proposition 3.1 (Conditional non-anticipation)**.** *Fix the auxiliary inputs: the encoder and its feature profile, the cached dex, the checkpoint (including its embedded* `strategy_knowledge`*), the frozen research and scout snapshots, our own team sets and, for search, the usage table and random seed. Suppose the request and protocol prefix supplied at decision t contain only information available to our player by t. Then the neural features and the search agent's sampled worlds are measurable functions of h_t and those inputs. Two calls with the same request, prefix and inputs produce the same features, and the same distribution over sampled worlds, whatever happens later and whatever remains unrevealed.*

Proof. `observe` and `Tracker.feed` scan only the supplied prefix and merge only the supplied own request. Neither queries the opponent's private stream or any later log suffix. Feature construction is deterministic in the observation, the candidate string and the fixed inputs. The scout is a fixed function of a filtered public prefix. The search agent samples worlds from a distribution that depends on the tracker state, the usage table and the seed. Composition of measurable maps proves the claim. □

The premise matters. Passing an omniscient log to `observe` would invalidate it. The local simulator bridge therefore observes through the separate `getPlayerStreams().p1/p2` streams; its omniscient stream only drives the simulator [18, 21]. The proposition also does not certify an experiment's chronology. The checkpoint's `strategy_knowledge` and the scout are trained on earlier games, and a held-out evaluation must not include games those earlier models were fitted on.

Caveat. Inside each sampled world the search engine is *omniscient* about that world. Both players' sets are fixed and known to both simulated sides. Non-anticipation holds for the world-sampling step. The per-world matrix game, however, is a perfect-information model of an imperfect-information game. Section 11.7 analyses the bias this introduces.

3.4 How faithful the observation is

The parser covers a selected subset of protocol events. Missing condition data maps to full HP. Under the historical actor's `strategic-v1` profile, the state encoder still uses the *legacy* HP parser. A coloured public HP string such as `50/100y` fails to parse and becomes 1.0. The bug was fixed only for the `tactics-v1`, `rain-v1` and `opening-v1` profiles. In the last 60 ladder games, 16 of 2,509 opposing condition entries carried a colour suffix, affecting 8 of 523 decisions. The strategic prior itself uses the corrected parser, so the defect reaches the network's learned residual but not its prior. Historical observed opponent moves are stored as `moves_known` and used by the planner; `Features._mon` encodes `moves`. Perspective correctness therefore does not imply a sufficient representation.

Implementation: `observe`, `hp_fraction` and `legacy_hp_fraction` in `battle_state.py`; `Features` in `ml/features.py`; `search/tracker.py`; `simulator.cjs`.

4 Candidate actions and doubles constraints

4.1 Actions are complete player decisions

For an actionable request r, let $\mathcal{A}(r)$ be the list returned by `legal_choices(r)`. In doubles, one candidate is a complete pair $a = (u_1, u_2)$. Each u_i is a move slot with a target, a switch destination or a pass. Examples are

```
move 1 1, move 2 -1
move 1 2 mega, switch 4
```

Positive targets are opposing positions and negative targets are our own. The opponent's simultaneous choice is *not* part of a candidate. The enumerator takes the Cartesian product of the per-position options. It removes combinations that violate shared constraints, filters disabled or exhausted moves, respects visible trapping, and handles fainted positions. More than two active positions are rejected. Hidden restrictions can still make the server refuse a candidate, so $\mathcal{A}(r)$ is *request-visible admissibility*, not proof of acceptance.

Proposition 4.1 (Constraints enforced by joint enumeration)**.** *Every emitted doubles candidate has distinct switch destinations, at most one Mega-family activation and at most one Terastallization. In a forced-replacement request, the number of switching positions equals* $\min($*forced positions*, *available bench members*$)$*.*

Proof. Each product element is tested in turn. Its switch strings must form a set, so duplicates are rejected. Under forced switching, its switch count must equal the stated minimum. Elements with more than one Mega-family event or more than one Terastallization event are rejected. An emitted element has passed every predicate. □

The proposition covers these predicates only. Champions requests never offer Terastallization, so that branch is dormant. Team validation and other hidden mechanics are outside its scope.

4.2 Team preview and an exact symmetry

With n party members and k to bring, preview candidates are ordered selections, $|\mathcal{A}_{\text{preview}}| = n!/(n-k)!$, which is 360 for six choose four. The first two positions lead; the last two are the bench. Under `opening-v2` and every `strategic-*` profile except `strategic-turn-v1`, the action encoder canonicalizes the bench by sorting its indices before hashing. The representation is therefore exactly

invariant to swapping the two bench positions. The actor still scores all 360 candidates, so each plan appears twice.

Proposition 4.2 (Bench symmetry halves the preview space)**.** *Let $\sim$ identify two preview candidates that agree in their ordered leads and in their bench as a set. For six choose four this gives 180 classes of exactly two candidates each. If the features, and hence the logits, are constant on each class, then:*

1. *the class probabilities $\Pi(C) = \sum_{a \in C} \pi(a)$ are a softmax over the 180 class logits, at the same temperature;*
2. *the PPO ratio of any candidate equals the ratio of its class;*
3. *the candidate-level entropy is $H(\pi) = H(\Pi) + \log 2$.*

Proof. There are $6 \cdot 5 = 30$ ordered lead pairs. Each leaves four members, from which $\binom{4}{2} = 6$ bench sets can be formed, so there are 180 classes. Each bench set has exactly two orderings, which accounts for all $180 \cdot 2 = 360$ candidates. Let ℓ_C be the shared logit. Then $\pi(a) = e^{\ell_C/\tau} / \sum_{C'} 2e^{\ell_{C'}/\tau}$ for $a \in C$, so $\Pi(C) = 2\pi(a) = e^{\ell_C/\tau} / \sum_{C'} e^{\ell_{C'}/\tau}$. The ratio $\pi_\theta(a)/\pi_{\text{old}}(a) = \Pi_\theta(C)/\Pi_{\text{old}}(C)$ because the factor 2 cancels. Finally, $H(\pi) = -\sum_C 2\pi_C \log \pi_C = -\sum_C \Pi(C) \log(\Pi(C)/2) = H(\Pi) + \log 2$. □

Interpretation. The duplication is harmless for learning. Ratios are unchanged, and the entropy bonus is shifted by a constant whose gradient is zero. Statements such as "the policy chooses among 360 previews" should still say 180 distinct plans. Lead order is kept on purpose: it fixes which Pokémon occupies the left and right positions.

4.3 Why joint scoring rather than a product policy

Sampling each position independently gives $\pi(u_1, u_2 \mid x) = p(u_1 \mid x)q(u_2 \mid x)$. That family cannot express coordination.

Proposition 4.3 (A coordination a product policy cannot represent)**.** *With two options A, B per position, no unmasked product distribution gives positive mass to both (A, A) and (B, B) and zero mass to both (A, B) and (B, A).*

Proof. Positive diagonal mass forces $p(A), q(A), p(B), q(B) > 0$, so the off-diagonal products are positive too. Equivalently, every product table has rank one and determinant zero. The required table has determinant $\pi(A, A)\pi(B, B) > 0$. □

The implemented alternative scores each feasible complete candidate directly. Masking a product policy with joint constraints, or factorizing it autoregressively, would also introduce dependence. Feature collisions and finite capacity still limit which joint distributions the network can express.

4.4 Two enumerators

The search engine enumerates candidates separately, inside the simulator, in `sideChoices` of `search/engine.cjs`. Its pruning differs from $\mathcal{A}(r)$:

- Damaging moves aimed at the partner are removed.
- Targets in empty foe slots are removed while the other foe is alive.

- Mega is forced when available under the default policy. When both actives can Mega, plain variants are also enumerated so valid combinations can activate exactly one, rather than dropping every double attack.

Our own live choices still start from $\mathcal{A}(r)$ and are pruned in Python. The engine enumeration is used for the opponent and for our forced-replacement look-ahead. The both-Mega enumeration defect recorded by the second edition was repaired in `b12314c`; Appendix C tracks its status.

Implementation: `legal_choices` in `battle_state.py`; bench canonicalization in `Features.action` (`ml/features.py`); `_prune_ours` in `search/agent.py`; `sideChoices` in `search/engine.cjs`.

5 Features, profiles and the strategic prior

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17 describe the later models, experiments and deployment.

5.1 Fixed-dimensional vectors for variable objects

The state vector has $d_s = 384$ coordinates and each candidate's action vector has $d_a = 192$. A named feature f with value v_f is added by signed hashing:

$$\phi_j \mathrel{+}= \sigma(f)\, v_f, \qquad j = H(f) \bmod (d - 16), \qquad \sigma(f) \in \{-1, +1\}. \tag{3}$$

Both H and σ come from an eight-byte BLAKE2b digest: the first four bytes, little-endian, give H, and the parity of the fifth byte gives the sign. The last sixteen coordinates are reserved for dense numbers. That leaves 368 hashed state buckets and 176 hashed action buckets. Feature names are namespaced by role and position: species, item, ability, HP, status, boosts, stats, types, moves with their PP ratio, volatiles, field and side conditions. The last sixteen public move events are added with weight $1/\sqrt{i+1}$, counting back from the newest. A research prior adds `meta/<species>` frequencies. It is built from the top 32 team-sheet frequencies, overlaid by the top 32 usage entries. Dense state slots hold turn progress $\min(\text{turn}/50, 2)$, total own HP divided by six, and $|\mathcal{A}|/1000$.

The whole vector is clipped to $[-5, 5]$. After clipping, the decision code adds the scout's top-three move probabilities (Section 13). The *stored* state therefore can leave the clipped interval.

Proposition 5.1 (Hashing loses distinctions)**.** *A signed hash map from m categorical indicator coordinates into $d' < m$ buckets ($d' = d - 16$ here) is not injective on $\mathbb{R}^m$. Any later function of the hashed vector, including clipping, cannot restore the lost distinctions.*

Proof. The map is linear with rank at most $d' < m$, so its null space is nontrivial. Two inputs that differ by a null-space vector hash to the same output, and any later function maps equal inputs to equal outputs. □

This is a capacity statement, not a claim that two particular battle states collide. Expected-collision results for feature hashing assume random hash functions [33]; a fixed digest over a realized vocabulary does not inherit them automatically.

5.2 Feature profiles

A checkpoint stores a `feature_profile`. Training excludes episodes recorded under any other profile, so a profile is a versioned input semantics. The schema number alone is not enough to identify the inputs. Nineteen profiles exist; Table 3 lists the families.

Profile family	What it changes
`legacy`, `weather-v1`	Original features and the power/type prior of Section 5.4; weather-aware move data.
`tactics-v1`, `rain-v1`, `mechanics-v1`	Damage-based turn priors and rain-team mechanics. `tactics-v1` and `rain-v1`, with `opening-v1`, are the only profiles that use the corrected coloured-HP parser.
`preview-*`, `lineup-v1`, `opening-*`, `mega-v1`	Preview priors (coverage, lead pressure, exposure) and exact prospective Mega stats under Champions rules.
`strategic-v1` (**deployed**)	Scenario-evaluation prior for turns and previews (Section 5.3); bench canonicalization.
`strategic-v2..v5`	More mechanics, preview plan summaries, Mega uncertainty. v2–v4 were evaluated and rejected; the v5 experiment was paused before a completed gate (Section 14).

Table 3: Feature-profile families. Only `strategic-v1` is deployed.

5.3 The deployed strategic prior

The final action coordinate holds a scalar prior $q_0(x, a)$. Under `strategic-v1` it comes from a one-turn *scenario evaluation* in `ml/strategy.py`, which has four steps.

Opponent hypotheses. For each live opposing active Pokémon, candidate moves are drawn from the disclosed sheet if there is one. Otherwise they are the observed moves plus the five most frequent executed moves for the species in the checkpoint's `strategy_knowledge`. That table holds counts from 766 earlier games. Each move gets the weight

$$w = 2 + \sqrt{\text{count}} + \min(3, \text{recent}), \tag{4}$$

with Protect multiplied by 1.5 below 40% HP, by 0.65 otherwise, and by 0.3 if it was used on the previous turn. The two strongest attacks and one support move are kept. A single-target attack is split over targets: weight 1 goes to the target with the largest damage relative to its remaining HP ($\text{damage}/\max(0.2, h)$), and 0.45 to the others. Two further hypotheses are added when they apply:

- a switch-in from the revealed bench, with weight $0.12 \sum w$;
- a 50/50 Mega-forme split, when the stone is disclosed and the side's Mega is unspent.

Joint scenarios. Per-position hypotheses are multiplied across the two opposing positions. Impossible combinations, such as two Megas or two switches into one reserve, are removed. The six most probable scenarios are kept and renormalized to probabilities p_k.

Deterministic resolution. For our candidate a and scenario k, `simulate` resolves one turn in priority and speed order, including Trick Room. It accounts for Protect, Wide Guard, redirection, Helping Hand, Fake Out, weather, terrain, Intimidate on entry, drain, recoil and Focus Sash. Damage uses the standard formula

$$\text{dmg} = \left(\frac{22\,\text{BP}\,A/D}{50} + 2\right) \cdot \text{STAB} \cdot E \cdot \text{burn} \cdot \text{weather} \cdot 0.75^{[\text{spread}]} \cdot 0.925 \cdot \text{acc} \,/\, \text{maxHP}, \tag{5}$$

where unknown stats are approximated as base $+35$ with stage multipliers, and unknown maximum HP as base HP $+90$. The constant 0.925 is the mean damage roll. The outcome value is

$$v_k(a) = \sum_{\text{foes}} [\Delta h + 0.8\,\mathbf{1}_{\text{KO}}] - \sum_{\text{own}} [\Delta h(1+0.2\rho) + (0.9+\rho)\mathbf{1}_{\text{KO}}] - 0.08\,\#\text{Protect} - 0.10\,\#\text{switch} + \ldots, \tag{6}$$

where $\rho \in [0, 0.5]$ is a scarcity bonus for a Pokémon that is the only good answer to some opponent. Smaller terms reward a switch-in's next-turn threat and penalize spending the Mega.

Risk-sensitive aggregation and squashing. If speeds tie, each v_k averages both orders. The turn score is

$$s(a) = 3\Big[(1-\kappa)\sum_k p_k v_k(a) + \kappa \min_k v_k(a)\Big], \qquad \kappa = \text{clip}\left(0.15 + 0.05(R_{\text{own}} - R_{\text{foe}}), 0.05, 0.25\right), \tag{7}$$

where R_{own} sums our HP fractions. R_{foe} sums the revealed foes' HP fractions and counts each unseen foe as one. Preview candidates use `opening_score` instead. It takes the better of no-Mega and one-Mega modes over coverage, lead pressure and exposure, adds terms for rain dependency, Intimidate, Fake Out and Trick Room, and doubles the total. Finally,

$$q_0(a) = g(s(a)), \qquad g(s) = \frac{4s}{4+|s|}. \tag{8}$$

Proposition 5.2 (Properties of the risk aggregate and the squash)**.**

1. *For any scenario values,* $\min_k v_k \le (1-\kappa)\sum_k p_k v_k + \kappa \min_k v_k \le \sum_k p_k v_k$*. The aggregate is monotone in each* v_k*, and adding a constant* c *to every* v_k *adds* c *to it.*
2. *The function* g *is odd, strictly increasing and a bijection from* $\mathbb{R}$ *onto* $(-4, 4)$*. It satisfies* $0 < g'(s) = 16/(4+|s|)^2 \le 1$*, so it is 1-Lipschitz. Its ranking of candidates equals the ranking by* s*.*

Proof. (1) Since $\min_k v_k \le \sum_k p_k v_k$, a convex combination of the two lies between them. Each part is monotone and translation-equivariant, and so is their combination. (2) For $s \ge 0$, $g(s) = 4s/(4+s)$ has derivative $16/(4+s)^2 > 0$; oddness extends this to $s < 0$. The function is continuous at 0, and $g(s) \to \pm 4$ as $s \to \pm\infty$. The derivative is largest at $s = 0$, where it equals 1, which gives the Lipschitz bound. A strictly increasing map preserves order. □

Interpretation. The squash replaces a hard clip, which had flattened 288 of the 360 preview candidates into one tied value (`docs/strategic-validation.md`). The pessimism weight κ *increases* when we are ahead on resources: the prior becomes more risk-averse when there is more to lose. The final `clip` to $[-4, 4]$ in the source is redundant after g.

Example 5.3 (A strategic prior, end to end)**.** *Three scenarios have values* $(0.9, 0.4, -0.6)$ *and probabilities* $(0.5, 0.3, 0.2)$*, with equal resources, so* $\kappa = 0.15$*. The expectation is* 0.45*, the minimum is* -0.6*, and* $s = 3[0.85(0.45) + 0.15(-0.6)] = 0.8775$*. The squash gives* $q_0 = 4(0.8775)/4.8775 \approx 0.720$*. At the deployed temperature* $\tau = 0.25$ *(Section 6), this prior adds* $0.720/0.25 \approx 2.88$ *to the effective logit before any learned residual.*

5.4 The legacy power prior

Earlier profiles used the rough pressure term

$$p(m, j) = \frac{\mathrm{BP}(m)}{100}\,\mathrm{acc}(m)\,\mathrm{STAB}(m)\,E(m, j), \tag{9}$$

summed over live targets, negated against allies, minus 0.4 per switch, plus 0.15 for Protect, Detect or Wide Guard, and clipped to $[-4, 4]$.

Example 5.4 (A legacy prior)**.** *Base power 80, accuracy 1, STAB 1.5 and a super-effective multiplier of 2 give* $0.8 \cdot 1.5 \cdot 2 = 2.4$*. Paired with Protect the candidate scores 2.55; paired with a switch it scores 2.0. These numbers are preferences, not HP damage or win probabilities. At preview this legacy prior is zero; the strategic prior is not.*

The legacy term still feeds a dense action slot, but it is not the deployed prior. Neither prior models full speed control, stat spreads, most items, or the opponent's best response. Search (Section 11) addresses exactly that last omission.

Implementation: `Features.add/state/action` in `ml/features.py`; `hypotheses`, `prepare`, `simulate`, `turn_score` and `opening_score` in `ml/strategy.py`; `damage` in `ml/tactics.py`; `preview_score` in `ml/opening.py`; `species_prior` in `ml/research.py`.

6 The action-conditioned actor–critic

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17 describe the later models, experiments and deployment.

6.1 Exact architecture

Let $x \in \mathbb{R}^{384}$ be the state vector and $z_a \in \mathbb{R}^{192}$ the vector for candidate a. The deployed `concat-v1` network computes

$$e_s = \tanh(W_s x + b_s) \in \mathbb{R}^{128}, \qquad e_a = \tanh(W_a z_a + b_a) \in \mathbb{R}^{96}, \tag{10}$$

$$\ell_\theta(x, a) = w_\pi^\top \tanh(W_\pi [e_s; e_a] + b_\pi) + c_\pi + q_0(x, a), \tag{11}$$

$$V_\theta(x) = w_V^\top \tanh(W_V e_s + b_V) + c_V, \tag{12}$$

with an actor hidden width of 96 and a critic hidden width of 64. The prior $q_0(x, a) = z_a[191]$ is the last action coordinate (Section 5). It enters as a fixed residual and has no learned weight. The critic sees only the state. Figure 3 shows the data flow.

The `matchup-v1` migration loads every old tensor and zero-initializes u, so migrated logits are exactly unchanged. That variant passed a development panel (70 vs. 55 of 96) but tied its final panel 126–126 of 200, and was rejected. The deployed checkpoint (revision `893e35e3`, 113 updates) is `concat-v1` with profile `strategic-v1`. It uses a feedforward summary of history: no recurrence, no attention, and no search inside the network.

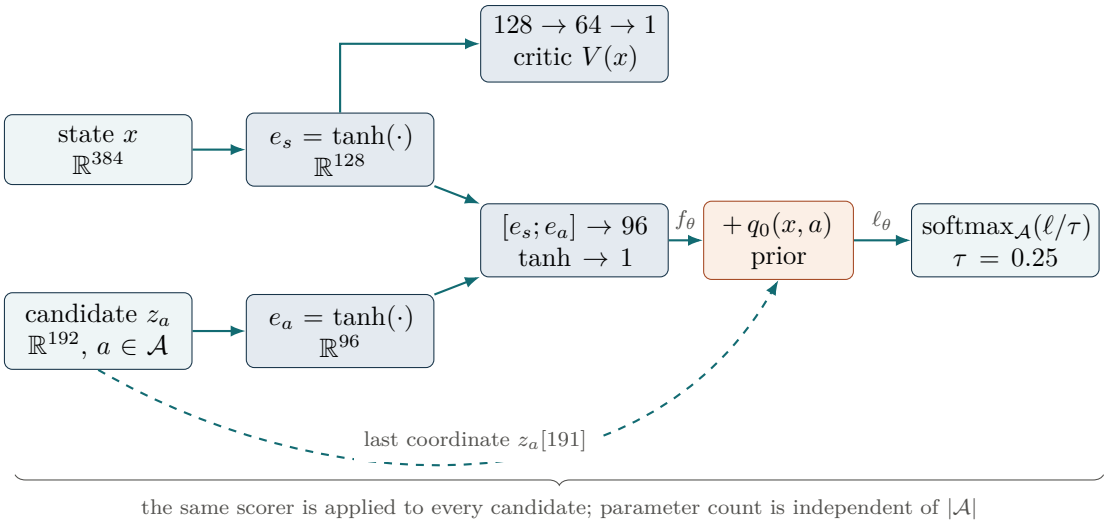


Figure 3: The deployed actor–critic. The final actor layer is initialized to zero, so a fresh actor equals softmax(q_0/τ). The optional `matchup-v1` variant adds a bilinear term $u^\top(e_a \odot Me_s)$ to the logit.

Module	Shape	Parameters
State encoder	$384 \to 128$	49,280
Action encoder	$192 \to 96$	18,528
Actor head	$224 \to 96 \to 1$	21,697
Critic head	$128 \to 64 \to 1$	8,321
`concat-v1` total (**deployed**)		**97,826**
`matchup-v1` extra: M $(128 \to 96)$, u $(96 \to 1)$	no bias	$12{,}288 + 96$
`matchup-v1` total (rejected)		110,210
Scout (Section 13)	$384 \to 96 \to 938$	127,946

Table 4: Parameter counts, checked by instantiating the classes. The scout's output width equals the move vocabulary of the live format.

6.2 The tempered masked policy

A checkpoint carries two temperatures in $[0.25, 2]$: one for turns and one for previews. The deployed value is $\tau = 0.25$ for both. For a nonempty candidate set the policy is

$$\pi_\theta(a \mid x, \mathcal{A}) = \begin{cases} \dfrac{\exp(\ell_\theta(x,a)/\tau)}{\sum_{u\in\mathcal{A}} \exp(\ell_\theta(x,u)/\tau)}, & a \in \mathcal{A}, \\ 0, & a \notin \mathcal{A}. \end{cases} \tag{13}$$

The same τ is used for sampling, for stored log-probabilities, and for the training loss. Inference scores only real candidates. Training pads variable candidate sets and fills padded logits with the most negative finite float. After division by $\tau < 1$ that sentinel overflows to $-\infty$ in float32. The categorical distribution handles this exactly, so padded probabilities are exactly zero.

Proposition 6.1 (Normalization, support and score function)**.** *For finite logits, $\tau > 0$ and a nonempty, parameter-independent set $\mathcal{A}$, the policy (13) sums to one, vanishes outside $\mathcal{A}$, and for $a \in \mathcal{A}$ satisfies*

$$\nabla_\theta \log \pi_\theta(a \mid x, \mathcal{A}) = \frac{1}{\tau}\Big(\nabla_\theta \ell_\theta(x,a) - \sum_{u\in\mathcal{A}} \pi_\theta(u \mid x, \mathcal{A}) \nabla_\theta \ell_\theta(x,u)\Big). \tag{14}$$

Proof. The denominator is a finite sum of positive terms, so the admissible probabilities are positive and sum to one; the others are zero by definition. The logarithm is $\ell_\theta(x,a)/\tau - \log \sum_u \exp(\ell_\theta(x,u)/\tau)$. Differentiating the log-sum-exp gives the π-weighted average of $\nabla \ell_\theta/\tau$. □

Interpretation. A low temperature multiplies every logit gradient by $1/\tau = 4$ and sharpens the policy. Turn-temperature promotion was itself the project's largest single measured improvement: the same weights at $\tau = 0.25$ won 129 of 200 paired games against 95 at the previous setting.

Example 6.2 (Temperature changes the same preferences)**.** *For logits $(2.55, 2.00, 0.30)$, $\tau = 1$ gives probabilities $(0.594, 0.343, 0.063)$. The deployed $\tau = 0.25$ gives $(0.900, 0.100, 0.0001)$. Greedy play picks the first candidate in both cases. These numbers are selection probabilities, not win probabilities.*

6.3 The learned residual as a tilt of the prior

Fix an input and candidate set. Split the logit into the prior q_0 and the learned residual $f_\theta = \ell_\theta - q_0$, and let $q_\tau = \mathrm{softmax}(q_0/\tau)$. Then

$$\pi_\theta(a) = \frac{q_\tau(a)\, e^{f_\theta(a)/\tau}}{Z}, \qquad Z = \sum_u q_\tau(u) e^{f_\theta(u)/\tau}. \tag{15}$$

Proposition 6.3 (Variational form with temperature)**.** *For a strictly positive q_τ and finite f on a finite set, the distribution $p^* \propto q_\tau e^{f/\tau}$ is the unique maximizer of $\sum_a p(a) f(a) - \tau D_{\mathrm{KL}}(p \| q_\tau)$ over probability vectors p.*

Proof. From $\log p^* = \log q_\tau + f/\tau - \log Z$,

$$\sum_a p(a) f(a) - \tau D_{\mathrm{KL}}(p \| q_\tau) = \tau \log Z - \tau D_{\mathrm{KL}}(p \| p^*).$$

The right side is maximized, uniquely, where the KL divergence vanishes, that is at $p = p^*$. □

Interpretation. The policy behaves as if it maximized the learned residual against a KL penalty toward the tempered prior, with penalty strength τ. At $\tau = 0.25$ the prior is sharpened four-fold and the residual is also amplified four-fold. The trainer never optimizes this objective explicitly, and a large enough residual can overturn the prior completely.

Permutation equivariance. The scorer is applied independently to each candidate. Permuting the candidate rows therefore permutes the logits and probabilities in the same way. With a unique maximum, the greedy choice is independent of list order. Exact ties fall back to the first index.

Implementation: `ActorCritic`, `Model.temperature`, `Model.predict` and `enable_matchups` in `ml/model.py`; `Brain.decide` in `ml/brain.py`.

7 The data recorded for every match

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17 describe the later models, experiments and deployment.

A learning system is only as trustworthy as its records. The recorder answers four questions for every decision:

- What did the player know?
- Which candidates were legal?
- With which probabilities, under which exact checkpoint, was one sampled?
- What happened afterwards?

The current format is *recorder 3*.

7.1 Anatomy of an episode

Figure 4 shows one episode as stored in the `episodes` table. An episode is a JSON document of four parts: a header, the conditions of collection, the decision steps, and a terminal section.

7.2 Tables

The store is one SQLite database in write-ahead-log mode. Its tables are listed in Table 5. Jobs carry a persistent identity, status, attempt count and a lease, which is how the three lanes of Section 9 share work safely.

7.3 What makes a record usable for learning

Before any gradient step, each candidate episode passes `trajectory_issues`. An episode is excluded, not repaired, if any check fails:

1. It is not complete, has another format, revision, schema or feature profile, or its outcome is outside $\{-1, 0, 1\}$.
2. Any ladder step was proposed but never submitted.
3. Any array has the wrong shape or a non-finite value, or any stored log-probability exceeds 10^{-6}.

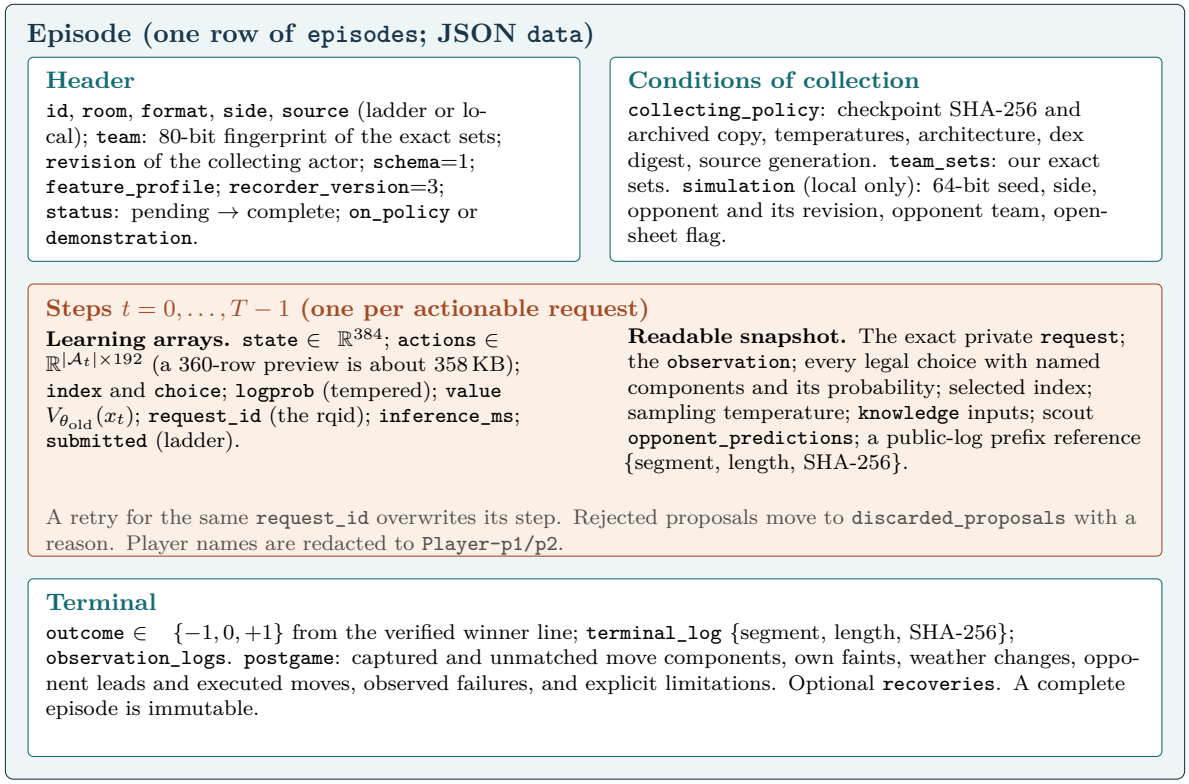


Figure 4: The recorder-3 episode. Learning uses the orange arrays; auditing and replay use the snapshot. For the last 20 ladder games the median document was 1.30 MB (range 0.73–2.06 MB). In a 7-step game, feature arrays took 750 KB and snapshots 440 KB of 1.20 MB.

Table	Contents	Rows (M-C)
episodes	Header columns plus the JSON document above	14,137 complete
jobs	Collector, learner, feed and evaluator jobs with leases	—
public_battles	Sanitized public logs, unique by digest	12,573
scout_samples	Pre-turn public vectors with executed-move labels	192,835
research_teams, documents	Attributed tournament sets and prose	—
team_metadata, feed_state	Team versions; daily feed bookkeeping	—

Table 5: SQLite schema. Counts are a read-only snapshot taken on 9 October 2026 at about 20:15 UTC, while collection was still running (database 18.4 GB). Of the complete M-C episodes, 854 are recorded ladder games (353–501) and 13,283 are local self-play games (9,010–4,273); there are no ties. The recorded ladder count also includes browser games played in model mode after the 847-game campaign stopped.

4. The snapshot's probabilities do not sum to one, or $p[\text{index}] \neq e^{\text{logprob}}$ (relative tolerance 10^{-5}).
5. The snapshot's choices differ from `legal_choices` of its own stored request.
6. A log-prefix digest fails, a step contains future or terminal lines, a request is credited twice, or the terminal log lacks a win or tie line.

A second check recomputes every stored log-probability under the archived collecting checkpoint (Section 8.5). Excluded episodes are never marked consumed, so a persistent defect stays visible in later batch reports rather than disappearing silently.

7.4 Games played by search

A browser game decided by search writes no `episodes` row. Its actions were not sampled from π_θ, so storing them as PPO samples would give false importance ratios (Section 8.4). The runner logs each decision, with its request, choice and search statistics, to a private per-run ledger. At the end of the game, the sanitized public log enters `public_battles` as an `own-live-game` and yields scout labels.

7.5 Storage lifecycle and budget

Data has a bounded lifetime, shown in Figure 5. Each candidate freezes the inference inputs it was evaluated with: research teams, documents, species and move support, and scout weights. These are stored as a content-addressed, read-only snapshot. Completed snapshots are gzip-archived every 900 s, with byte-count and SHA-256 verification, so they stay reproducible. A storage budget limits optional work.

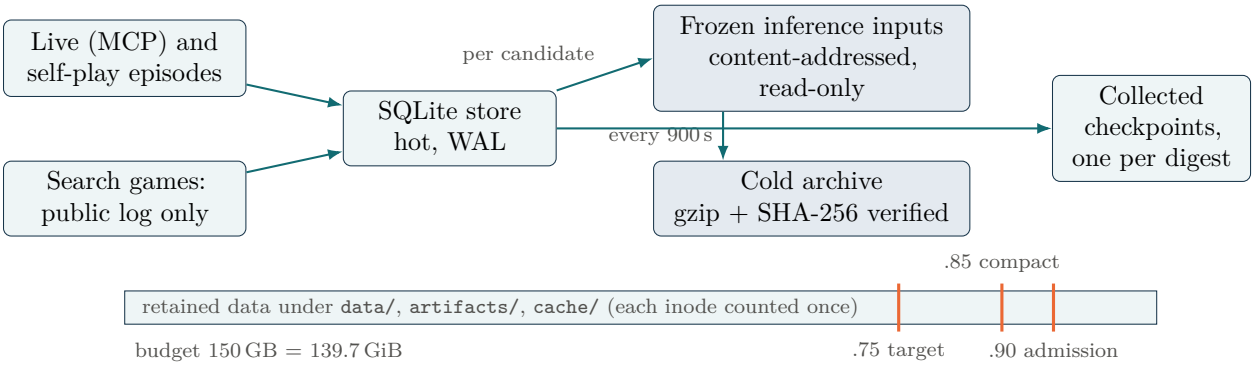


Figure 5: Data lifecycle and the storage budget. Above 90% of the budget, collection, learning and evaluation defer. Above 85%, finished candidates' input snapshots are compacted down to 75%. Collecting checkpoints are kept once per digest, so every recorded likelihood can be recomputed.

Implementation: `ml/storage.py` (schema, fingerprint, collected checkpoints); `ml/brain.py` and `ml/recording.py` (steps and snapshots); `ml/postgame.py`; `ml/data_quality.py`; `ml/input_snapshot.py`; `ml/data_budget.py`; `docs/cold-input-archiving.md`; `docs/model-storage.md`.

8 Learning from outcomes: policy gradients, GAE and PPO

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17

describe the later models, experiments and deployment.

8.1 Reward and objective

For a completed episode with T recorded decisions, let $Y \in \{-1, 0, 1\}$ be loss, tie or win from our side. The reward is terminal only: $r_t = 0$ for $t < T - 1$ and $r_{T-1} = Y$. With $\gamma = 0.99$,

$$G_t = \sum_{j=t}^{T-1} \gamma^{j-t} r_j = \gamma^{T-1-t} Y. \tag{16}$$

With $\gamma = 1$, $\mathbb{E}[Y] = \mathbb{P}(\text{win}) - \mathbb{P}(\text{loss}) = 2\mathbb{P}(\text{win}) + \mathbb{P}(\text{tie}) - 1$. When ties are absent, as in every recorded M-C game, maximizing $\mathbb{E}[Y]$ is the same as maximizing win probability. With $\gamma < 1$, the same outcome is worth less after a longer game, so training mildly prefers shorter wins and longer losses. The promotion gate counts raw wins. The two criteria are related but not identical.

For a fixed policy, the history-conditioned value, action value and advantage are $V^\pi(h_t) = \mathbb{E}_\pi[G_t \mid h_t]$, $Q^\pi(h_t, a) = \mathbb{E}_\pi[G_t \mid h_t, a_t = a]$ and $A^\pi = Q^\pi - V^\pi$. The critic approximates V^π through compressed features. Its output is unbounded and is not a calibrated win probability.

8.2 The policy-gradient identity on histories

Theorem 8.1 (Finite-horizon likelihood-ratio gradient)**.** *Assume a finite trajectory space and a finite horizon T, padding terminated trajectories with zero-reward absorbing steps. Assume bounded rewards, a differentiable policy with parameter-independent support on each history, and environment and opponent kernels that do not depend on θ. Let $J(\theta) = \mathbb{E}_\theta[\sum_{t<T} \gamma^t r_t]$. Then*

$$\nabla_\theta J(\theta) = \mathbb{E}_\theta\Big[\sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t \mid h_t)\Big], \tag{17}$$

and G_t may be replaced by $G_t - B(h_t)$ for any action-independent baseline B without changing the expectation.

Proof. A trajectory's probability factors into initial, transition and policy terms. Only the policy terms depend on θ, so $\nabla \log p_\theta(\tau) = \sum_t \nabla \log \pi_\theta(a_t \mid h_t)$. Differentiating the finite expectation gives $\mathbb{E}[R(\tau)\nabla \log p_\theta(\tau)]$. A reward earned before a_t is determined by h_t, and $\mathbb{E}[\nabla \log \pi_\theta(a_t \mid h_t) \mid h_t] = \sum_a \nabla \pi_\theta(a \mid h_t) = \nabla 1 = 0$. Those cross terms therefore vanish, which leaves $\sum_{j \geq t} \gamma^j r_j = \gamma^t G_t$. The same zero-mean identity removes the baseline. □

Interpretation. An action gains probability when the outcome that follows it is better than expected for that situation. The theorem needs no Markov assumption on the hashed features, because it is stated on histories. It does need the recorded action to have actually been sampled from π_θ. This is why search decisions are never PPO data.

Caveat. The implemented surrogate averages decision samples equally and omits the γ^t factor. It is therefore a standard heuristic related to Equation (17), not an unbiased estimator of ∇J [25, 26, 29]. Opponent kernels are not fixed in self-play either: the `self` opponent is a frozen snapshot of the incumbent, which is fixed within a batch but changes after each promotion.

8.3 Generalized advantage estimation

Using the collecting critic's stored values V_t, and $V_T = 0$,

$$\delta_t = r_t + \gamma V_{t+1} - V_t, \qquad \widehat{A}_t = \delta_t + \gamma\lambda \widehat{A}_{t+1}, \qquad \widehat{A}_T = 0, \qquad \lambda = 0.95, \tag{18}$$

$$\widehat{R}_t = \widehat{A}_t + V_t. \tag{19}$$

Figure 6 shows how the recursion runs backward through one episode.

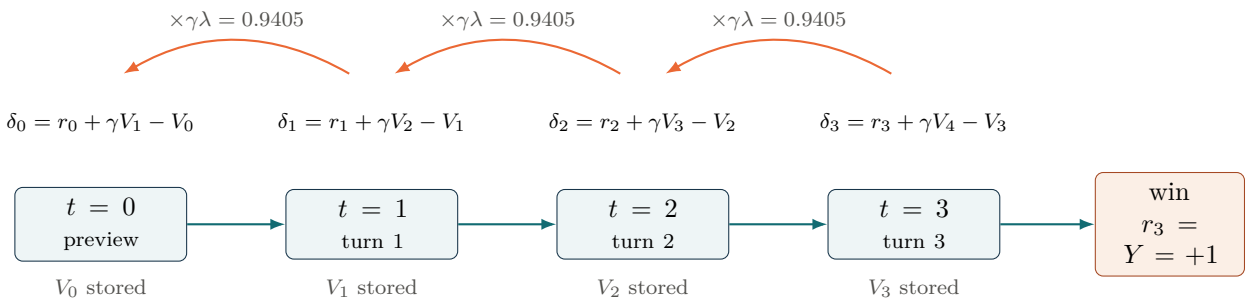


Figure 6: GAE on a four-decision episode with a terminal win. Only the last residual contains the reward. Each earlier advantage adds its own residual to the next advantage, discounted by $\gamma\lambda$.

Proposition 8.2 (GAE expansion and the Monte Carlo endpoint)**.** *The recurrence* (19) *gives* $\widehat{A}_t = \sum_{j=t}^{T-1} (\gamma\lambda)^{j-t}\delta_j$. *If* $\lambda = 1$ *and* $V_T = 0$, *then* $\widehat{A}_t = G_t - V_t$.

Proof. Unroll the recurrence backward from $\widehat{A}_T = 0$, or argue by induction from $t = T - 1$. With $\lambda = 1$, the $+\gamma V_{j+1}$ in each residual cancels the $-V_{j+1}$ in the next one after discounting. What remains is the discounted rewards, $-V_t$, and $\gamma^{T-t} V_T = 0$. □

Example 8.3 (Two decisions ending in a win)**.** *With* $(V_0, V_1) = (0.20, 0.30)$ *and* $(r_0, r_1) = (0, 1)$*:* $\delta_1 = 0.70$, $\delta_0 = 0.99(0.30) - 0.20 = 0.097$, $\widehat{A}_1 = 0.70$, *and* $\widehat{A}_0 = 0.097 + 0.9405(0.70) = 0.75535$. *The critic targets are* $(0.95535, 1.00)$*; the Monte Carlo target for* $t = 0$ *would be 0.99.*

8.4 Stored probabilities and importance ratios

For a step collected by revision θ_{old}, PPO uses

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid x_t, \mathcal{A}_t)}{\pi_{\theta_{\text{old}}}(a_t \mid x_t, \mathcal{A}_t)} = \exp\left(\log \pi_\theta(a_t \mid x_t, \mathcal{A}_t) - \text{storedLogProb}_t\right), \tag{20}$$

with both policies tempered as in Equation (13).

Proposition 8.4 (Conditional importance identity)**.** *At a fixed stored input and candidate set, if the old policy* p *is positive wherever the new policy* q *is, then* $\mathbb{E}_{a\sim p}[(q(a)/p(a))f(a)] = \mathbb{E}_{a\sim q}[f(a)]$ *for every real* f.

Proof. Write the left side as $\sum_a p(a)\, q(a) f(a)/p(a)$ and cancel $p(a)$ on its support. The support assumption means no term with positive q-mass is lost. □

This corrects the action choice at fixed inputs only, not the whole trajectory distribution. It is wrong when the action was really chosen by greedy play, a human, search, or an unknown policy. Rejections add a further subtlety. The stored likelihood is the probability of the *proposal*. If hidden restrictions reject some proposals and only accepted ones survive, the executed action follows the proposal policy conditioned on acceptance. The recorder therefore keeps rejected proposals separately, and accepts a step only after its submission is confirmed.

8.5 Collecting-likelihood validation

Before training, the learner reloads the parent (collecting) weights and recomputes every stored step's tempered log-probability. Any episode with a step where the recomputed and stored values differ by more than $\eta = 10^{-4}$ is excluded.

Proposition 8.5 (What the likelihood check guarantees)**.** *Suppose every retained step satisfies* $|\log \pi_{\theta_{\text{old}}}(a_t) - \text{storedLogProb}_t| \le \eta$*, with the left side recomputed from the stored inputs. Then for every* θ *the ratio used in training equals the exact ratio times* e^{ϵ_t} *with* $|\epsilon_t| \le \eta$*. For* $\eta = 10^{-4}$*, the relative ratio error is at most* $e^{\eta} - 1 \approx 1.00005 \times 10^{-4}$*.*

Proof. Write $\epsilon_t = \log \pi_{\theta_{\text{old}}}(a_t) - \text{storedLogProb}_t$. Then $\exp(\log \pi_\theta - \text{storedLogProb}) = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})e^{\epsilon_t}$, and $|\epsilon_t| \le \eta$ by assumption. □

Interpretation. The check certifies more than arithmetic. A trajectory whose features, candidate list, checkpoint or temperature changed after recording would fail it. The six most recent learner jobs had a maximum discrepancy of 1.9×10^{-6}. The check cannot certify that the recorded inputs were themselves a faithful observation; Section 3 covers that separately.

8.6 The optimized PPO loss

Advantages are standardized once over the whole batch, before any epoch: $\widetilde{A}_t = (\widehat{A}_t - \overline{A})/(s_A + 10^{-8})$ when the batch has more than one sample and $s_A > 10^{-6}$. Here s_A is the population standard deviation. Each minibatch $\mathcal{B}$ of size B also carries phase weights $\omega_t = w_t / \overline{w}_{\mathcal{B}}$, where w_t is the preview training weight ($1 \le w \le 16$) for preview steps and 1 otherwise. The minimized loss is

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{t \in \mathcal{B}} \omega_t \Big[-\min(\rho_t \widetilde{A}_t, \text{clip}(\rho_t, 0.8, 1.2)\widetilde{A}_t) + 0.5(V_\theta(x_t) - \widehat{R}_t)^2 - c_H \, H(\pi_\theta(\cdot \mid x_t, \mathcal{A}_t)) \Big]. \quad (21)$$

The entropy is that of the tempered distribution, and the value loss is not clipped. The deployed configuration is: entropy coefficient $c_H = 0.01$; preview weight 1 (so $\omega_t \equiv 1$); Adam with learning rate $3 \cdot 10^{-4}$; minibatches of 32 drawn by a seeded permutation; four epochs; global gradient-norm clip 0.5; target KL 0.03. A non-finite loss aborts before any checkpoint is written. Each training call produces a new revision UUID and increments the update counter once.

Proposition 8.6 (What clipping bounds)**.** *For a fixed advantage* A *and positive ratio* ρ*,*

$$\min(\rho A, \text{clip}(\rho, 1-\epsilon, 1+\epsilon)A) = \begin{cases} A \min(\rho, 1+\epsilon), & A \ge 0, \\ A \max(\rho, 1-\epsilon), & A < 0, \end{cases}$$

and this is never larger than the unclipped term ρA*.*

Proof. For $A \geq 0$ multiplication by A preserves order, so the minimum picks the smaller of ρ and $\text{clip}(\rho)$. That is $\min(\rho, 1+\epsilon)$, because clipping from below never lowers ρ. For $A < 0$ the order reverses. The minimum picks the larger of the two, which is $\max(\rho, 1-\epsilon)$. Taking a minimum with ρA cannot exceed ρA. □

Example 8.7 (A clipped update)**.** *If* $\pi_{\text{old}}(a) = 0.20$*,* $\pi_\theta(a) = 0.30$ *and* $A = 0.70$*, then* $\rho = 1.5$*. The unclipped term is 1.05; the clipped term is* $1.2(0.70) = 0.84$*. With* $A = -0.70$ *and* $\rho = 0.5$ *the clipped term is* $0.8(-0.70) = -0.56$*, not* -0.35*.*

The proposition does not keep every ratio within $[0.8, 1.2]$, does not enforce a KL constraint, and does not lower-bound win-rate improvement [24, 26]. The KL stop below is the implementation's coarse trust region.

8.7 The KL early stop

After each full epoch, the learner evaluates the whole batch at the current parameters and computes

$$\widehat{D_{\text{KL}}} = \frac{1}{N}\sum_{i=1}^{N}\left[(\rho_i - 1) - \log \rho_i\right], \tag{22}$$

where ρ_i is the ratio at the stored action. If $\widehat{D_{\text{KL}}} > 0.03$, the remaining epochs are skipped. The updates from the epoch that crossed the threshold are *kept*.

Proposition 8.8 (The KL estimator)**.** *Fix a state, an old policy* p *and a new policy* q *with the same support, and let* $\rho(a) = q(a)/p(a)$*. Then each term* $(\rho - 1) - \log \rho$ *is non-negative, and* $\mathbb{E}_{a\sim p}[(\rho(a) - 1) - \log \rho(a)] = D_{\text{KL}}(p\|q)$*. Consequently, if the stored actions were sampled from the old policy, Equation (22) is an unbiased estimate of the average* $D_{\text{KL}}(\pi_{\text{old}}\|\pi_\theta)$ *over the batch's stored states.*

Proof. For $u > 0$, $u - 1 - \log u \geq 0$, with equality only at $u = 1$, because $\log u \leq u - 1$. Next, $\mathbb{E}_p[\rho] = \sum_a q(a) = 1$, so $\mathbb{E}_p[\rho - 1] = 0$, and $\mathbb{E}_p[-\log \rho] = \sum_a p(a)\log(p(a)/q(a)) = D_{\text{KL}}(p\|q)$. Averaging over states with their stored actions preserves unbiasedness, by linearity. □

Caveat. The estimator is unbiased for $D_{\text{KL}}(\text{old}\|\text{new})$ (23). The check runs only between epochs, so the returned candidate can exceed 0.03 by up to one epoch's movement. Recent production batches stopped at final-epoch values between 0.012 and 0.017, so none triggered the stop. The statistic is still computed on the same samples used for the gradient, so it is a training-set diagnostic rather than a held-out one.

8.8 Demonstrations

For explicitly labelled demonstrations $(x_t, \mathcal{A}_t, a_t^*)$, a separate training call minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{B}\sum_t \omega_t \log \pi_\theta(a_t^* \mid x_t, \mathcal{A}_t), \tag{23}$$

the weighted conditional log-likelihood. There is no ratio, entropy or value term, and no collecting-likelihood check. The shared state encoder still changes, so the critic moves too. Imitating a weak teacher does not make the learner an expert.

Implementation: `Model.train` (eligibility, GAE, likelihood validation, standardization, loss, KL stop, saving) in `ml/model.py`; `RolloutBatcher` in `ml/batching.py`; `trajectory_issues` in `ml/data_quality.py`; `LearningConfig` in `ml/continuous.py`.

9 The training pipeline on CPU and Apple MPS

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17 describe the later models, experiments and deployment.

9.1 From a game to a deployed checkpoint

Figure 7 traces one unit of experience through the pipeline. Each arrow is a durable hand-off: a SQLite row, a job, a file written by atomic replacement, or a lease. A crash at any point therefore leaves either the old state or the new one, and the next supervisor tick resumes from it.

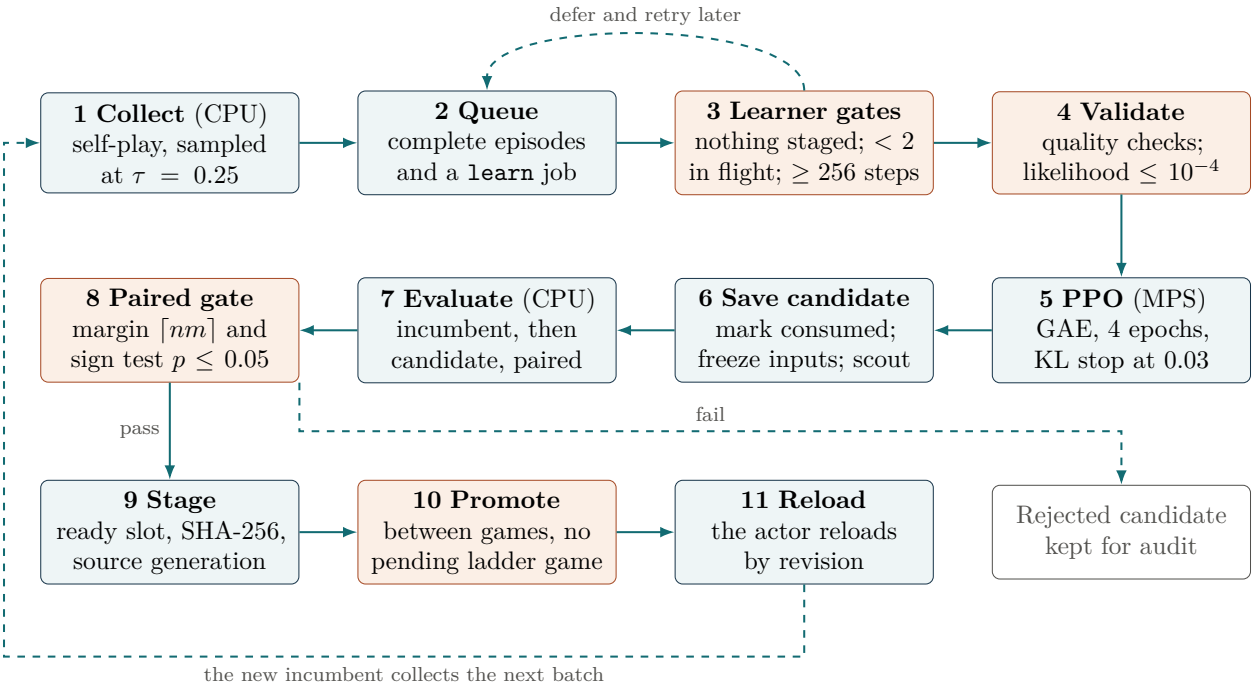


Figure 7: The lifecycle of experience. Orange boxes are gates that can stop the flow. Steps 1 and 7 run on the CPU, step 5 on the MPS GPU. Promotion (step 10) is called by the player between games, never by a worker. Episodes are marked consumed when the candidate is saved, before it is evaluated, so "trained" means *used for a candidate*, not *deployed*.

Collection. The collector snapshots the incumbent and freezes its inference inputs. It then plays `simulation_games` games (default 24) in waves of spawned one-thread processes. Each process drives the official `BattleStream` through `simulator.cjs` and observes only its own player stream. The `ladder-v1` curriculum cycles the opponent through

$$(\text{self},\ \text{tactical},\ \text{heuristic},\ \text{self},\ \text{tactical},\ \text{random},\ \text{heuristic},\ \text{tactical}),$$

with index $(\lfloor i/|\text{teams}| \rfloor + \text{seed}) \bmod 8$, so one opponent completes a sweep of the team pool before the next takes over. In each cycle that is 2/8 self-play against the frozen incumbent, 3/8 tactical, 2/8 heuristic and 1/8 random. Open team sheets are drawn with probability 0.08. Sides alternate as $(i + \lfloor i/|\text{teams}| \rfloor) \bmod 2$. Seeds are 64-bit, split into four 16-bit words. An earlier encoding used only 16 bits and was retired.

Learning. The learner applies the gates of step 3, the validation of step 4 and the loss of Section 8. It runs on the device chosen by `backend_for`. In automatic mode MPS is chosen only if its benchmarked time is at most 0.85 of the CPU time. Any device error retrains the same batch from the incumbent on the CPU. For the strategic profiles the learner then refreshes the candidate's embedded `strategy_knowledge` with new ladder postgame counts. It also trains the public-move scout on the same lane.

Evaluation. Incumbent and candidate are frozen CPU snapshots. They play n games each (60 in the deployed configuration; default 20), on seeds $s_0 + 100000 + i$ derived from the job identity. Actions are sampled at the checkpoint temperature. Both panels face the same frozen `self` snapshot and the same frozen inputs. Panels resume after interruption. The evaluator abandons a plan if the source generation, parent revision or candidate hash has changed.

9.2 Three lanes on one laptop

Figure 8 shows how the work is shared out in time. A supervisor under launchd `KeepAlive` wakes every 15 s. It reloads the autopilot settings, samples resources, enqueues daily feeds and collection, and starts at most one child per lane, running `python -m ml.worker -lane` with the lane name (`collector`, `learner` or `evaluator`). Each child takes its own lock and claims jobs atomically with `BEGIN IMMEDIATE`. A job's lease is its time budget plus 60 s. A job can be claimed while its attempt count is below three, and a deferral gives its attempt back, so deferrals do not exhaust a job. A child exits after its time budget, and the next tick replaces it.

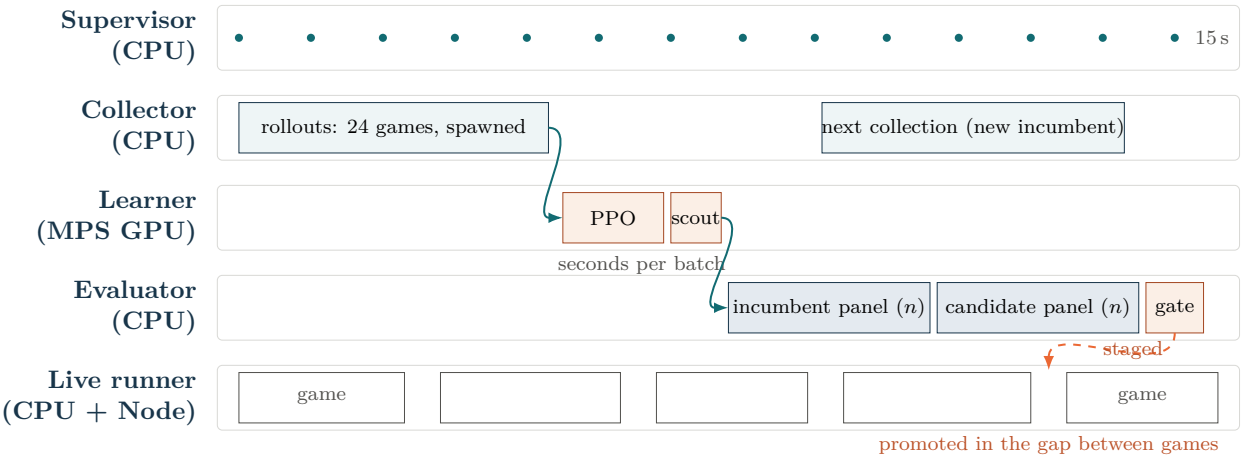


Figure 8: Schematic timeline of the lanes; durations are not to scale. The collector and evaluator share one pool of simulator slots. Within the default budget of four, the evaluator gets half (calibrated locally to three) and the collector the rest. The learner owns the GPU. The live runner is outside the pipeline and only ever consumes a staged candidate at a game boundary.

9.3 Resource admission

Every lane re-samples the host before starting work and at minibatch checkpoints. With C logical cores, a measured CPU load of u percent, available memory F GiB, r resident workers and an

estimated m MiB per worker, the allowed simulator worker count is

$$W_{\text{mem}} = \max(0, \lfloor (F - F_{\min})1024/m \rfloor + r), \qquad W_{\text{cpu}} = \max(0, \lfloor (u_{\max} - u)C/100 \rfloor), \tag{24}$$

$$W = \mathbf{1}[\text{healthy}] \cdot \max(0, \min[W_{\max},\, C - C_{\text{res}},\, W_{\text{mem}},\, W_{\text{cpu}}]), \tag{25}$$

where *healthy* means $F \geq F_{\min}$, $u < u_{\max}$, swap-out below its limit, no critical memory pressure (macOS `kern.memorystatus_vm_pressure_level` flag value 4), and at least 2 GiB of free disk. "Yellow" pressure (flag value 2) is allowed; only "red" blocks. The code defaults are $C_{\text{res}} = 3$, $F_{\min} = 2$ GiB, $W_{\max} = 4$, $u_{\max} = 75\%$, $m = 400$ MiB and swap-out 32 MiB/s. The deployment uses a modestly higher CPU ceiling and a calibrated swap limit. Optional work also stops above 90% of the storage budget (Figure 5).

Proposition 9.1 (Headroom invariants)**.** *Whenever $W > 0$:*

1. *at most $C - C_{\text{res}}$ simulator workers run, so C_{res} cores stay free of simulator workers;*
2. *if each* new *worker uses at most m MiB, launching $W - r$ of them leaves at least $F_{\min}$ GiB available;*
3. *if each worker adds at most $100/C$ percentage points of load (one core), the projected load $u + 100W/C$ stays at or below $u_{\max}$.*

Proof. (1) $W \leq C - C_{\text{res}}$. (2) $W - r \leq \lfloor (F - F_{\min})1024/m \rfloor$, so $(W - r)m/1024 \leq F - F_{\min}$. (3) $W \leq \lfloor (u_{\max} - u)C/100 \rfloor$ gives $100W/C \leq u_{\max} - u$. □

Interpretation. The memory term credits resident workers, whose memory is already counted in F. The CPU term does not credit them, even though their load is already in u. It is therefore conservative and can under-admit while workers are running. These are cooperative limits enforced by the application, not OS quotas. Another service can still take the headroom between samples, and the 5-second re-sampling only bounds how long that can go unnoticed.

9.4 The MPS lane

The learner calls `torch.mps.set_per_process_memory_fraction(0.12)`, which caps its Metal allocations at 12% of the device's recommended working-set size [22]. Its thread count is limited and it runs at `nice` 10. A GPU duty fraction $d \in (0, 1]$ throttles training. After each optimizer step the learner synchronizes with Metal, measures the step's busy time e_k, and sleeps

$$s_k = e_k\Big(\frac{1}{d} - 1\Big). \tag{26}$$

Proposition 9.2 (Duty cycle)**.** *Under this rule each step, and therefore any union of steps, has busy fraction exactly d: $\sum_k e_k / \sum_k (e_k + s_k) = d$.*

Proof. $e_k + s_k = e_k/d$, so $\sum_k (e_k + s_k) = \sum_k e_k/d$. □

The proposition concerns wall-clock time inside the training loop, measured after synchronization. It is not a statement about instantaneous GPU utilization, which the application cannot observe. The default is $d = 0.5$. For these small cached batches, half duty cost several times the full-duty time: 34.99 vs. 7.45 ms per update in Table 6, which is more than the factor of two Proposition 9.2 accounts for. The deployment therefore runs at full duty and relies on the memory cap and the admission guards (`docs/offline-ppo-followup.md`).

9.5 The tensor cache

Rebuilding Python feature lists dominated training time, so batches are converted to tensors once. With N steps and at most C candidates per step, the resident footprint in float32 is $N(384 \cdot 4 + C(192 \cdot 4 + 1) + 8)$ bytes. The extra byte per candidate is its mask entry and the 8 bytes per step hold the index. The batch is kept on the device if this is at most 2^{27} bytes (128 MiB). Otherwise a bounded CPU LRU cache transfers minibatches one at a time.

Example 9.3 (When a batch stays resident)**.** *A batch containing a 360-candidate preview has* $C = 360$*, so each step costs* $1536 + 360 \cdot 769 + 8 = 278{,}384$ *bytes. The residency limit is* $\lfloor 2^{27}/278{,}384 \rfloor = 482$ *steps. Recent production batches of 404 and 415 steps were resident; a 617-step batch used the CPU cache.*

64-decision batch, up to 360 choices	Before cache (ms/update)	After cache (ms/update)
CPU	43.08	14.08
MPS, 50% duty	121.61	34.99
MPS, full duty	44.06	7.45

Table 6: Bounded M5 timing windows from `docs/m5-learning-orchestration-research.md`. They were measured while other workloads ran, so they are diagnostic timings rather than randomized comparisons. A raw kernel benchmark favoured MPS by only 1.057 vs. 1.512 ms; most of the speedup came from caching.

9.6 Safe reloads of source code

The *source generation* is a SHA-256 over the entry points, `simulator.cjs`, the lock file and all `ml/*.py` files. Rollouts refuse to run under a mismatched generation. On a change, the supervisor stops starting children, drains the running ones and exits with code 75 so that launchd restarts it under the new code. Evaluation plans and staged candidates record their generation, and promotion refuses a candidate evaluated under different code. A source change therefore invalidates evaluations; it never mixes code versions inside one comparison. On 8–9 October, 25 evaluations were abandoned for this reason and two because the parent had advanced.

Implementation: `ml/pipeline.py` (supervisor, slots), `ml/worker.py` (lanes, collection, learn, evaluate), `ml/resources.py` (`ResourcePolicy`, `BackgroundGuard`, `backend_for`), `ml/batching.py`, `ml/reload.py`, `ml/simulator.py`, `simulator.cjs`, `scripts/learning_service.py`.

10 Promotion: a paired test and what it can certify

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17 describe the later models, experiments and deployment.

10.1 The paired design

For game i the incumbent and the candidate face identical conditions: the same seed, side, opponent, opponent revision, learner team, opponent team, open-sheet flag, seed encoding and effective simulator seed. The gate rejects the run if any of these differ between the two panels. Let $X_i^I, X_i^C \in \{0, 1\}$ be the incumbent's and the candidate's wins in pair i. Pairs with $X_i^I = X_i^C$ are *concordant* and

carry no information about which policy is better. The informative pairs are the gains ($X^C = 1$, $X^I = 0$), counted by g, and the losses, counted by ℓ. The discordant total is $d = g + \ell$, and $W_C - W_I = g - \ell$. Figure 9 shows the idea.

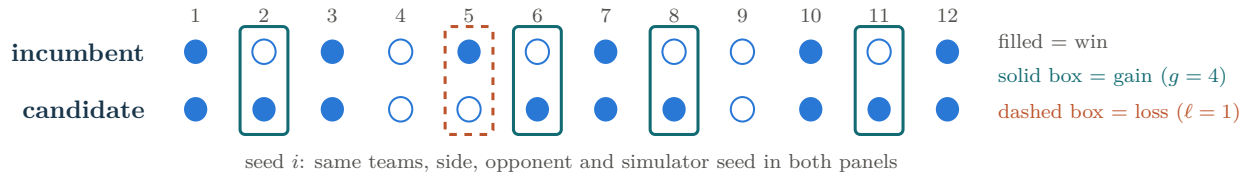


Figure 9: The paired design on twelve illustrative seeds. Only the five discordant seeds bear on the comparison. Here $g = 4$, $\ell = 1$ and $p = \mathbb{P}(\mathrm{Bin}(5, \frac{1}{2}) \geq 4) = 0.1875$, so this run would fail.

10.2 The gate

With n games per policy and margin m, a candidate passes when

$$\text{clean} \ \wedge \ (W_C - W_I \geq \lceil nm \rceil) \ \wedge \ p \leq 0.05, \qquad p = \sum_{k=g}^{d} \binom{d}{k} 2^{-d} \quad (p = 1 \text{ if } d = 0). \tag{27}$$

Clean means: the run is complete and fully paired; there are no unfinished games and no rejected actions; and every game has a winner or a tie. The defaults are $n = 20$, $m = 0.10$; the deployment uses $n = 60$, $m = 0.05$. The first edition's gate formula, $\max(1, \lfloor nm + 0.999 \rfloor)$ with no test, is obsolete.

Proposition 10.1 (Validity of the gate's test)**.** *Suppose the pairs are independent, and under the null hypothesis each discordant pair is equally likely to be a gain or a loss:* $\mathbb{P}(X^C = 1, X^I = 0) = \mathbb{P}(X^C = 0, X^I = 1)$ *for every pair. Then* $\mathbb{P}(p \leq \alpha) \leq \alpha$ *for every* $\alpha \in [0, 1]$*. In particular, the gate passes a candidate that is no better than the incumbent with probability at most 0.05.*

Proof. Condition on which pairs are discordant, and hence on d. Under the null, independence and the equal-probability condition make each discordant pair a gain with probability $\frac{1}{2}$, independently. So $g \mid d \sim \mathrm{Bin}(d, \frac{1}{2})$. The statistic p is the upper-tail probability of this exact distribution at the observed g. By the standard property of discrete tail probabilities, $\mathbb{P}(p \leq \alpha \mid d) \leq \alpha$. Averaging over d preserves the bound. The other gate conditions only remove passes. □

This is the exact conditional form of McNemar's test [12]. It needs neither a normal approximation nor equal marginal rates across pairs. Its null is "no systematic difference on the evaluated distribution", not "no difference anywhere".

Caveat. The source computes $\lceil nm \rceil$ in floating point. For some pairs this exceeds the exact ceiling by one; for example $100 \times 0.07 = 7.000000000000001$ rounds up to 8. The configurations actually used, $(20, 0.10)$, $(60, 0.05)$, $(96, 0.10)$, $(192, 0.10)$ and $(200, 0.05)$, are unaffected. A rational or integer form would remove the edge case.

10.3 Many gates: multiplicity and the winner's curse

One valid test is not the same as a valid campaign of tests. Between 8 October 18:18 UTC and 9 October 19:57 UTC the pipeline completed 129 paired 60-game gates, and one passed.

Discordant pairs d	Minimum gains g	Minimum net $g-\ell$	p at that g
4 or fewer	—	impossible	≥ 0.0625
5	5	5	0.0313
8	7	6	0.0352
10	9	8	0.0107
15	12	9	0.0176
20	15	10	0.0207
30	20	10	0.0494
40	26	12	0.0403

Table 7: What the sign test demands. With $n = 60$, $m = 0.05$ the margin requires a net of only $\lceil 3 \rceil = 3$, so the test is always the binding condition. The one production pass on 9 October was $38 \to 46$ wins with $g = 9$, $\ell = 1$, $p = 0.0107$.

Proposition 10.2 (Expected false promotions)**.** *Run K gates, each with type-I error at most α under its null. If every candidate is null, the expected number of passes is at most $K\alpha$, and the probability of at least one pass is at most* $\min(1, K\alpha)$*. Testing each at α/K (Bonferroni) bounds the family-wise error by α.*

Proof. The expected number of passes is the sum of the individual pass probabilities, each at most α under the null. The union bound gives the second claim and, with α/K, the third. □

With $K = 129$ and $\alpha = 0.05$, up to 6.45 false passes are possible even if no candidate is better, so a single pass is not evidence that PPO is improving the actor. The exact sign test is discrete and conservative, so the true expectation is somewhat lower, but the conclusion stands. A Bonferroni threshold would be $0.05/129 \approx 3.9 \times 10^{-4}$; the observed $p = 0.0107$ does not meet it. The project's own history shows the *winner's curse* directly. `matchup-v1` passed development ($p = 0.0068$) and then tied its independent final 126–126. `strategic-v4` reached $p = 0.008$ on a development panel and then went 139–136 on a fresh confirmation ($p = 0.39$). Selection on one panel inflates that panel's apparent effect. An independent, prospectively specified final panel is the remedy, and the project's experiment runner (`scripts/train_opening.py`) uses one.

10.4 What the gate does not measure

At the 9 October baseline this gate evaluated the *sampled self-play actor* against scripted opponents and a frozen copy of itself. Live turns, however, are decided by search, and the actor only chooses team previews, greedily. A candidate that passes therefore improves a decision system that is not the one deployed for turns. Its effect on live play runs through the preview, and through the data the next candidates are trained on. The later search gates and live A/B infrastructure explicitly address this construct-validity gap (Sections 15 and 16).

10.5 A distribution-free certificate (proposed, not implemented)

A lower confidence bound on the improvement would be a stronger deployment rule than a test.

Theorem 10.3 (Paired Hoeffding lower bound)**.** *Fix both decision systems before evaluation. Draw n evaluation units independently from a declared target distribution, and in unit i run both systems on matching conditions. Let $D_i = X_i^C - X_i^I \in [-1, 1]$ have common mean Δ, with $\overline{D} = n^{-1}\sum_i D_i$. Then, for $\delta \in (0, 1)$,*

$$\mathbb{P}\Big(\Delta \geq \overline{D} - \sqrt{2\log(1/\delta)/n}\Big) \geq 1 - \delta. \tag{28}$$

Proof. Hoeffding's inequality for independent variables in intervals of width 2 gives $\mathbb{P}(\overline{D} - \Delta \geq \eta) \leq \exp(-2n^2\eta^2/(4n)) = \exp(-n\eta^2/2)$ [6]. Set $\eta = \sqrt{2\log(1/\delta)/n}$ and take complements. □

Example 10.4 (How much evidence a margin is)**.** *At $n = 60$ and $\delta = 0.05$ the radius is $\sqrt{2\log 20/60} \approx 0.316$. The production pass, with $\overline{D} = 8/60 \approx 0.133$, has lower bound -0.18, so the certificate does not establish improvement. A radius of 0.10 needs $n \geq \lceil 2\log 20/0.01 \rceil = 600$ independent units. The strategic-v1 final, $\overline{D} = 19/160 \approx 0.119$ with radius 0.193, is also inconclusive under this bound, although its exact test gave $p = 0.0072$. Hoeffding ignores that most pairs are concordant. A variance-adaptive or exact discordant-pair interval is sharper, and is the right next step.*

Dependence *within* a pair is allowed; independence *across* units is required. If both sides of a seed are played, the seed is the unit. If games cluster by opponent team, the team is the unit: Section 14 re-tests the search result at that level. For K fixed candidates, use δ/K per candidate.

Implementation: `paired_gate`, `stage` and `promote_ready` in `ml/promotion.py`; `evaluate` in `ml/worker.py`; preregistered panels in `scripts/train_opening.py`; `docs/checkpoint-validation.md`.

11 Decision-time search

The historical actor's prior looked one turn ahead against a handful of guessed opponent moves. The deployed search replaces that approximate resolver with the official simulator. It samples complete hypothetical opposing teams, plays every pairing of our and the opponent's joint actions for one turn, and solves the resulting matrix game. Figure 10 shows the procedure.

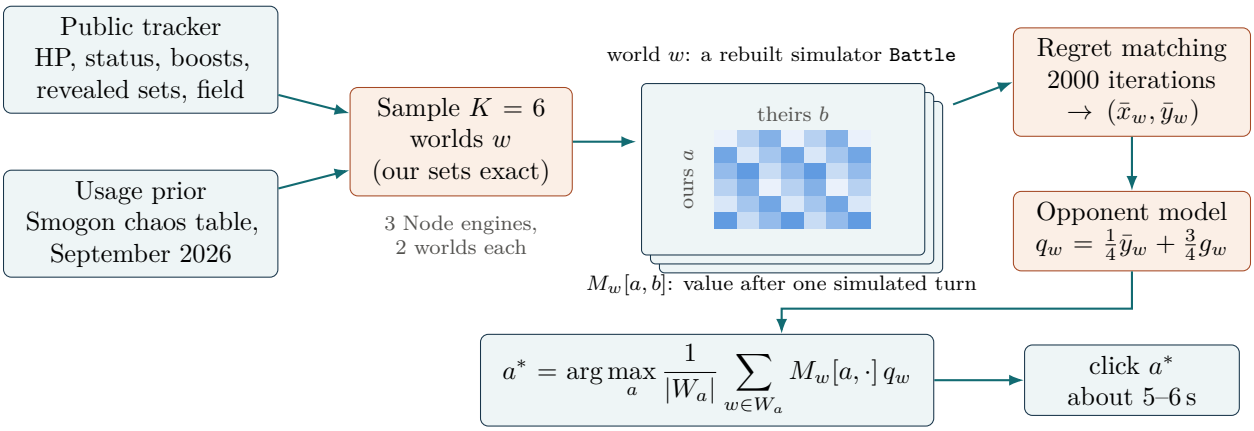


Figure 10: The search agent. Each world fixes complete hypothetical opposing sets and copies all public state. Screening keeps at most 24 of our joint actions and 20 of the opponent's. The payoff matrix of a world therefore needs at most 480 one-turn simulations, plus up to $4|\mathcal{B}| + 4|\mathcal{A}|$ for screening. Six initial worlds may be extended by six on close decisions within the adaptive time budget. The C5 leaf and selective continuation modify the basic one-turn game. Team preview uses a separate learned model.

11.1 Public tracker

`search/tracker.py` reads only our own player stream. For each Pokémon it tracks:

- HP fraction (exact HP when the denominator is not 100), status, sleep and toxic counters, boosts and volatiles;

- revealed moves, item (or its loss) and ability;
- Mega state, last move, and the turn it switched in.

For the field it tracks weather, terrain, pseudo-weather and side conditions, each with its start turn. A condition's remaining duration is estimated as the base duration minus elapsed turns. If that would be non-positive, the extended duration is assumed. Open team sheets are parsed from `|showteam|` lines.

11.2 Sampling hidden sets from usage statistics

The prior is Smogon's monthly *chaos* usage table for the format (September 2026, all ratings, about 1.6 million battles) [28]. For each usage entry e, which is a species or a Mega forme, the table gives a raw count c_e and normalized frequencies $P_e(\cdot)$ that sum to one over each category. The categories are moves, items, abilities and the 60 most common nature-and-stat-point spreads.

Lemma 11.1 (Forme posterior under conditional independence)**.** *Suppose an opposing Pokémon's entry is e with prior probability proportional to c_e. Suppose also that, given e, the event "move m is in the moveset" has probability $\pi_e(m)$ independently across moves. Then after observing the revealed set R, $\mathbb{P}(e \mid R) \propto c_e \prod_{m \in R} \pi_e(m)$. If, moreover, $\pi_e(m) = c\, P_e(m)$ for a constant c shared by all entries (with four move slots, $c \approx 4$), then*

$$\mathbb{P}(e \mid R) \propto c_e \prod_{m \in R} P_e(m). \tag{29}$$

Proof. The first statement is Bayes' rule with the likelihood factorized by the independence assumption. Under the proportionality assumption, the product becomes $c^{|R|} \prod_{m \in R} P_e(m)$. The factor $c^{|R|}$ is the same for every entry, so it cancels in the normalization. □

The implementation follows Equation (29), with three adjustments:

- unseen moves get a floor of 0.002 on the share scale, so a surprising reveal cannot zero out every entry;
- an observed non-stone item, or a spent Mega, removes the Mega entries;
- an observed Mega keeps only matching Mega entries.

Given e, the unrevealed moves are drawn without replacement in proportion to P_e. Item, ability and spread are each drawn from their own marginals. A Mega entry takes its most common item (assumed to be the stone) and the base forme's abilities. Unseen previewed Pokémon fill the remaining slots, drawn without replacement in proportion to species usage. When open sheets were exchanged, species, item, ability, moves and nature are copied exactly, and only stat points are sampled.

Caveat. Lemma 11.1 is exact only under its independence assumption. Real movesets have four slots, so move inclusions are negatively dependent. Item, ability and spread are correlated in real sets, and the `Teammates` table is loaded but unused. The Item Clause is not enforced across a sampled team. The result is a tractable proposal distribution, not a calibrated posterior over the opponent's team.

11.3 Rebuilding a world

Each world becomes a fresh simulator `Battle` built from both packed teams. Public state is then written onto it:

- formes, abilities, items, exact HP for us and round($h \cdot$ maxHP) for them, statuses with their counters, and our own PP;
- a whitelist of volatiles, such as Protect's stall counter, Substitute, Taunt, Encore, Disable and choice lock. The agent does not send last moves, so the engine's last-move restoration is unused, and the opponent's PP is never set. Recharge is now restored from public and private evidence;
- weather, terrain and side conditions with their estimated durations.

If a side has used its Mega, Mega Evolution is disabled for the whole side. Our side uses exact sets, HP and PP from our request.

11.4 The one-turn game in each world

Our candidates are the Python-legal choices $\mathcal{A}(r)$, pruned in two ways. Damaging and harmful single-target status moves at our partner are removed, and when a Mega is available only Mega-containing choices are kept. Each surviving choice is rewritten by move *identifier*, because a locked request lists only one move and slot numbers differ in the rebuilt battle. The opponent's set $\mathcal{B}$ comes from the engine's own enumeration (Section 4).

Screening. If $|\mathcal{B}| > 20$ or $|\mathcal{A}| > 24$, the engine takes four evenly spaced probe actions $P \subset \mathcal{A}$. It keeps the 20 opponent actions with the lowest mean value against P, which are the best for the opponent. It then keeps the 24 of our actions with the highest mean value against the first four kept replies.

Payoffs. For each remaining pair, the engine restores the world state, assigns a chance stream, submits both choices, runs one turn and evaluates the result (with selected second-turn corrections described below):

$$M_w[a, b] = V\big(\mathrm{Turn}(B_w; a, b; \xi_{w,b})\big). \tag{30}$$

If the simulator rejects any cell, that cell's whole row (our action) or column (their action) is removed.

Material evaluation and C5 leaf. The underlying hand evaluator scores a finished battle ± 30, or 0 for a tie. Otherwise $V = S(\text{us}) - S(\text{them}) + \text{TR}$, where

$$S = \sum_{\text{alive } p} \big[1 + 1.2h_p - \mathrm{pen}(p) + \mathbf{1}_{\text{active}}\,\beta(p)\big] + 0.10 \cdot \text{Tailwind turns} + 0.06 \sum \text{screen turns}. \tag{31}$$

Status penalties are: paralysis 0.30, freeze 0.50, poison 0.12, toxic 0.25, sleep $0.45\min(1, \text{turns}/2)$, and burn 0.45 if Attack exceeds Special Attack, else 0.12. The active-Pokémon term β rewards offensive, defensive and speed boosts and a Substitute. It penalizes Perish counts, confusion, Taunt, Encore, Yawn and Leech Seed. TR is the Trick Room term: it adds $0.10 \cdot \text{turns} \cdot 2\,\mathrm{clip}((\bar{v}_{\text{them}} - \bar{v}_{\text{us}})/40, -1, 1)$, where $\bar{v}$ is each side's mean Speed, so slower teams gain value under Trick Room.

The deployed C5 leaf transforms this hand score with Equation (39). Thus the matrix's V denotes the transformed leaf at the current cutoff, rather than raw material. When the world's

material score is below -1 and its preferred action Protects, the best Protect and non-Protect lines are selectively continued one more turn against the top three modeled replies. This is a targeted horizon correction, not full depth-two search. Speed-order constraints filter hidden-set draws.

11.5 Solving the matrix game

The engine runs simultaneous regret matching on the zero-sum game in which we maximize M [5]. With cumulative regrets R_t for us and C_t for the opponent,

$$x_t(i) \propto [R_t(i)]^+, \qquad y_t(j) \propto [C_t(j)]^+ \quad \text{(uniform if all} \le 0), \tag{32}$$

$$R_{t+1}(i) = R_t(i) + (My_t)_i - x_t^\top M y_t, \qquad C_{t+1}(j) = C_t(j) + x_t^\top M y_t - (x_t^\top M)_j, \tag{33}$$

for $T = 2000$ iterations. It returns the average strategies $\bar{x}$ and $\bar{y}$.

Proposition 11.2 (Average regret gives an approximate equilibrium)**.** *Let ε_1 and ε_2 be the two players' average external regrets after T rounds. Then $(\bar{x}, \bar{y})$ is an $(\varepsilon_1 + \varepsilon_2)$-equilibrium:* $\max_i (M\bar{y})_i - \min_j (\bar{x}^\top M)_j \le \varepsilon_1 + \varepsilon_2$. *For regret matching with payoff range Δ, $\varepsilon_1 \le \Delta\sqrt{|\mathcal{A}|/T}$ and $\varepsilon_2 \le \Delta\sqrt{|\mathcal{B}|/T}$.*

Proof. Let $\bar{v} = T^{-1}\sum_t x_t^\top M y_t$. By linearity, $T^{-1}\sum_t (My_t)_i = (M\bar{y})_i$, so our average regret is $\max_i (M\bar{y})_i - \bar{v} \le \varepsilon_1$. Likewise the opponent's is $\bar{v} - \min_j (\bar{x}^\top M)_j \le \varepsilon_2$. Adding the two gives the claim. Because $\min_j (\bar{x}^\top M)_j \le \bar{x}^\top M \bar{y} \le \max_i (M\bar{y})_i$, neither player can gain more than $\varepsilon_1 + \varepsilon_2$ by deviating. The regret rates are the standard bounds for regret matching [5, 35]. □

Interpretation. With $|\mathcal{A}| = 24$, $|\mathcal{B}| = 20$ and $T = 2000$, the worst-case gap is $\Delta(\sqrt{24} + \sqrt{20})/\sqrt{2000} \approx 0.21\Delta$. The original material leaf could span 60 with terminal values; the C5 transformed leaf lies in $(0, 4/.801)$, making the units different while the relative bound remains loose. In practice regret matching usually converges much faster than its worst case. The bound justifies $\bar{y}$ only as an approximate equilibrium, and only of this one-turn surrogate game.

11.6 The opponent model and the decision rule

The agent does *not* play $\bar{x}$. It treats the opponent as a mixture of two types:

$$q_w = \alpha\, \bar{y}_w + (1 - \alpha)\, g_w, \qquad g_w(b) \propto \exp\!\big(-(c_b - \min_{b'} c_{b'})/\tau_o\big), \qquad c_b = \frac{1}{|\mathcal{A}|}\sum_a M_w[a, b], \tag{34}$$

with $\alpha = \frac{1}{4}$ and $\tau_o = 1$ at the current cutoff ($\alpha = \frac{1}{2}$ at the second-edition baseline). One type plays the equilibrium. The other is a soft best response to our playing uniformly, a level-1 quantal response in the sense of McKelvey and Palfrey [11]. Each world then gives our action values $U_w(a) = (M_w q_w)_a$. Let W_a be the worlds in which a was valid, and W_{ok} the worlds that produced any result. The decision is

$$a^* = \arg\max_a \Big[\frac{1}{|W_a|}\sum_{w \in W_a} U_w(a) - 0.5\,\mathbf{1}\{|W_a| < |W_{\text{ok}}|\}\Big]. \tag{35}$$

Proposition 11.3 (Guarantee and exploitability of a best response)**.** *Let v^* be the value of a world's matrix game. For any opponent mixture y, a best response $a^* \in \arg\max_a (My)_a$ satisfies $(My)_{a^*} \ge v^*$. Its worst case, $\min_b M[a^*, b]$, can still be strictly below v^*.*

Proof. By the minimax theorem, $v^* = \min_{y'} \max_a (My')_a \le \max_a (My)_a$. For the second claim take matching pennies, $M = \left(\begin{smallmatrix} 1 & -1 \\ -1 & 1 \end{smallmatrix}\right)$, with $v^* = 0$ and $y = (\frac{1}{2}, \frac{1}{2})$. Every pure action is a best response, and each loses 1 to some reply. □

Interpretation. Against the *modelled* opponent the pure choice is never worse than the game's value. Against an opponent who anticipates it, a deterministic best response can be exploited, whereas $\bar{x}$ could not be. The design bets that ladder opponents neither compute this matrix nor exploit its argmax. The level-1 component also assumes they reason one step about our options. These are modelling choices. Subsequent C3, C9 and C12 experiments tested specific alternatives without validating a stronger replacement (Section 15).

11.7 Determinization and strategy fusion

Sampling worlds and solving each with full knowledge is *perfect-information Monte Carlo* (PIMC). It is known to suffer from *strategy fusion*, which means acting as if one could choose differently in worlds one cannot tell apart, and from *non-locality* [4]. Yet PIMC often plays well in practice [10]. Information-set search avoids fusion at a higher cost [2]. Here the turn decision avoids fusion at the root: Equation (35) commits to one a for all worlds. Fusion does enter forced-replacement scoring, which takes $\frac{1}{K} \sum_w \max_a U_w(a)$.

Proposition 11.4 (Forced-replacement scores are optimistic)**.** *For any values $U_w(a)$,*

$$\frac{1}{K} \sum_w \max_a U_w(a) \;\ge\; \max_a \frac{1}{K} \sum_w U_w(a),$$

with equality if and only if some single action maximizes U_w in every world.

Proof. For each a' and w, $\max_a U_w(a) \ge U_w(a')$. Averaging over w and then maximizing over a' gives the inequality. If a' maximizes in every world, the two sides coincide. Conversely, if no action does, then for every a' some world has a strict gap, so the average on the left exceeds every right-hand average. □

Interpretation. A replacement whose best follow-up depends on the hidden world is scored as if we will learn the world before moving. That inflates its value, so the rule favours flexible-looking replacements whose flexibility is not real. Within each world both simulated sides also know each other's sets, even when sheets were not shared, so the opponent model is omniscient about us.

11.8 Monte Carlo error

Each cell ordinarily uses one stochastic sample of accuracy, damage rolls and critical hits. Common random numbers now seed each retained opposing reply identically across our rows, unlike the second-edition baseline. Different actions can consume the stream differently, so equal seeds do not mean identical realized events. With six initial worlds the difference still contains world-sampling and within-cell chance noise. If the leading margin is below .5 and the initial batch took less than half of the 9-second budget, six fresh worlds are added. The engine timeout is 40 seconds, with a legal fallback before the ordinary 55-second turn allowance.

Proposition 11.5 (Ranking two actions)**.** *Suppose the per-world differences $D_w = U_w(a) - U_w(a')$ are independent, lie in an interval of width 2Δ, and have mean $\gamma > 0$. Then the average over K worlds ranks a' above a with probability at most* $\exp(-K\gamma^2/(2\Delta^2))$.

Proof. This is Hoeffding's inequality applied to the event $\overline{D} \leq 0$ with deviation γ [6]. □

With $K = 6$, the bound is useful only when γ is a large fraction of Δ; it does not establish reliable rankings for small gaps. CRN reduces variance when paired outcomes covary positively, which must be measured, not assumed universally. The implemented adaptive worlds reduce some observed ranking variation. C6's failed larger-budget candidates show that reduced noise need not translate into a detectable strength gain.

11.9 Preview, Mega and the learned components

Team preview uses the C4 matchup-value model by default; C14 compares that with a human-imitation argmax on the Mev team. An older simulator-value preview mode remains optional. Mega remains forced when a legal activation is available under the baseline, although double-Mega enumeration is repaired. The C7 species-avoidance option is implemented but was not promoted.

Value nets, learned action screening, learned opponent probabilities, public-history scouting and the shared root solver are separate opt-in paths. C1–C12 did not produce a further demonstrated live gain from these paths. The current configuration retains hand-based sigmoid leaves and probe screening; collecting their training labels does not imply that learned weights are in use. Section 15 reports the actual experiments, including C12's mixed root strategy, which remains disabled.

Implementation: `search/agent.py` (worlds, pruning, move-id rewriting, decision rule, forced replacements), `search/engine.cjs` (rebuild, enumeration, screening, payoffs, regret matching, evaluation), `search/priors.py`, `search/tracker.py`, `search/valuefeat.cjs`, `search/train_value.py`; `docs/search-agent.md`.

12 One live turn: browser transport, guards and viewer

12.1 The sequence

Figure 11 follows one turn from the server's request to the viewer's animation. The browser runner holds an account lock, so only one controller can drive the account. It reads the official client's state through `app.rooms[id]`, using the request, `battle.stepQueue`, the choice state and the preview order, and acts only by clicking visible controls. It opens no protocol socket of its own.

12.2 Freshness guards

Search takes seconds and the client keeps animating, so the request on screen can change while a decision is being computed. Three checks keep a stale decision from being clicked:

1. After deciding, the runner takes a new snapshot. The server rqid must be unchanged, the client must not already be waiting, and the choice must be legal under the new request. Only the rqid is compared, because the client mutates its own copy of the request.
2. A capture-phase listener on the document cancels any click in the room if `request.rqid` differs from the guarded rqid, or if the client is waiting.
3. Before each control the runner re-checks the rqid, and it aborts if the guard has fired.

Proposition 12.1 (Clicks act only on the decided request)**.** *Assume that the server assigns a fresh rqid within each battle room, and that the page runs JavaScript on a single-threaded event loop. Then every click that reaches the client's own handler does so while the displayed request has the rqid the decision was computed for, and while the client is not waiting.*

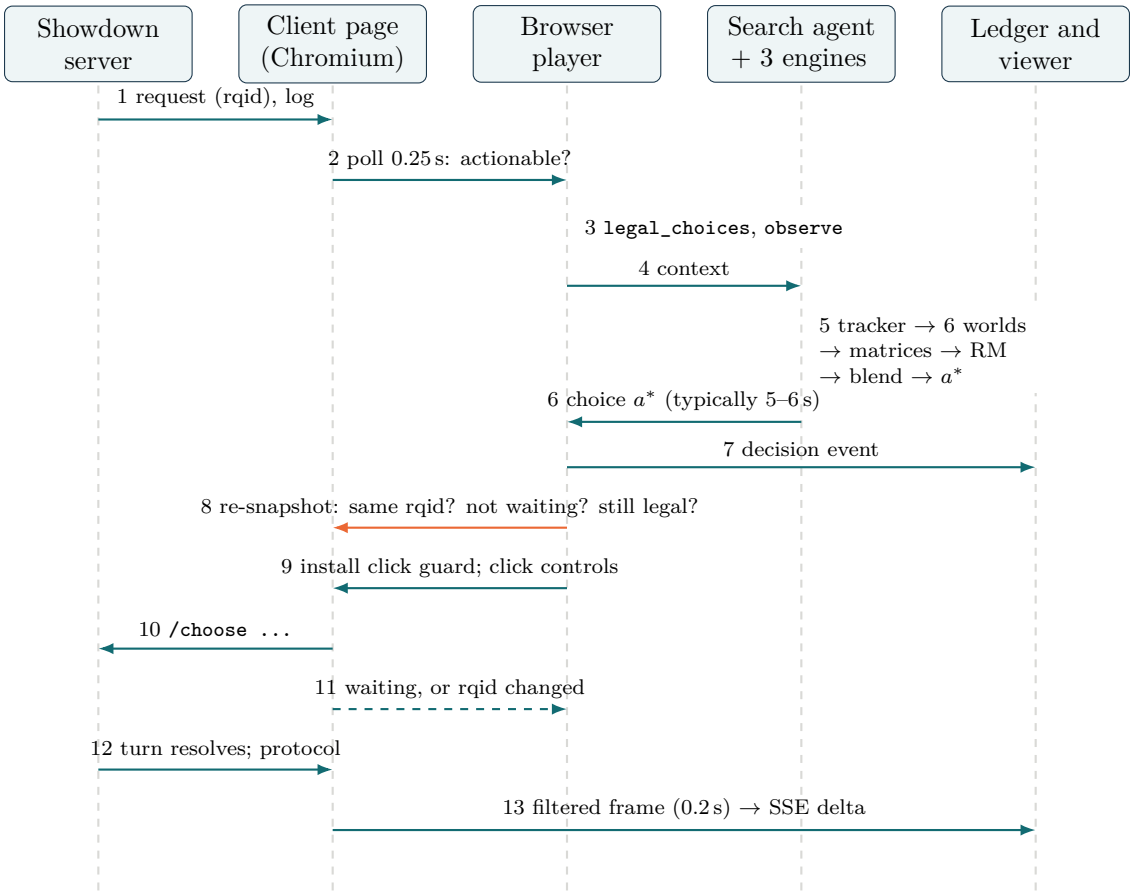


Figure 11: One live turn under search. Step 8 is the freshness check after the decision. Step 9 installs a capture-phase guard that cancels any click once the server's rqid has changed. Preview turns replace steps 4–6 with the selected pvalue or imitation preview model (originally the greedy actor). In recorded mode, steps 4–6 are `Brain.decide`.

Proof. A click event is dispatched to the capture listener and to the client's handler within one task of the event loop. A WebSocket message that would replace the request is processed in a separate task, so it cannot run between the guard's check and the handler. The guard lets the event proceed only if the rqid equals the decided one and the client is not waiting. Since rqids are fresh, equal rqids identify the same request. □

Caveat. The proposition protects individual clicks. A multi-click choice (Mega checkbox, move, target) can be cut off midway when the request changes, leaving a partial choice that the next request supersedes. Server acceptance is confirmed separately, by waiting up to 10 s for the client to show waiting or a new rqid. A new `|error|`, or an unchanged rqid with a changed fingerprint, counts as a rejection. At most three submissions are made per rqid, that is, two retries; a third rejection stops the run.

12.3 Recovery

Two recovery paths exist:

- **Recorded browser runs.** A run that restarts mid-game resubmits a saved, unsubmitted proposal only if its stored request equals the current one. If the saved proposal was already submitted, recorded mode stops rather than guessing, because the server's acknowledgement is ambiguous.
- **MCP path.** `Brain.resume` restores one compatible pending episode after a transport failure, with the same identity, steps and log-probabilities. It refuses ambiguous fragments or schema changes, and resubmits a saved action for the same rqid without resampling, with up to three attempts per room.

The first edition said that a crash loses the session with no mid-game reconstruction. That is no longer true, within these limits.

Proposition 12.2 (Scope of submission idempotence)**.** *Within one serialized session, a repeated submission for a cached request sends no second choice unless rejection handling removed the cache entry. Repeated finalization returns the cached terminal result. A complete stored episode is never overwritten.*

Proof. The choose path checks its cache key before selecting or sending, and finish returns an existing terminal result first. The store returns early when the stored identifier is already complete. □

This is not an exactly-once guarantee across processes. A send completing on the socket is not a server acknowledgement, and an external submission can still interleave.

12.4 The live viewer

The runner publishes filtered public protocol, with names replaced by "Your agent" and "Opponent", to an atomically replaced relay file every 0.2 s. A small server streams appended lines as server-sent events [34]. It sends a full snapshot on a room change or reconnect, and serves only over the tailnet with HTTPS. The client plays each turn in full. With two or more turns queued it switches to a faster speed. With four or more it jumps to the newest turn. A renderer stuck for 15 s is skipped forward. The viewer is for observation only and plays no part in decisions.

Implementation: `ml/browser.py` (`snapshot`, `context`, guards, `submit`, recovery, `watch`); `scripts/browser_play.py`; `scripts/live_watch.py`; `web/live-watch/`; `ml/live.py` (`resume`, transport recovery); `docs/browser-play.md`; `docs/live-watch.md`; Playwright [13].

13 Opponent-move scouting and team selection

Historical baseline (7–9 October). This section preserves the implemented actor-era design and its dated measurements. The PPO service is paused in the current search player; Sections 15–17 describe the later models, experiments and deployment.

13.1 The public-move scout

Public logs contain executed events, not private requests or a collecting policy's probabilities. The repository therefore uses them to train a classifier, the scout, and never fabricates PPO samples from them. `ingest_public` takes a snapshot of each player's public view at every turn boundary. A later executed move becomes a label for that snapshot only if all of the following hold:

- the actor was already on the field at the boundary;
- the actor is still unambiguous when the move happens;
- the move is in the dex;
- the move is not tagged `[from]` (called or forced).

Chat, authentication, HTML and private request lines are removed, and players are redacted. Incomplete games and orphan samples, whose source battle row is missing, are excluded.

Proposition 13.1 (Turn-boundary non-anticipation)**.** *If the supplied log is chronological, each retained scout vector is a function of the prefix up to its turn boundary and of an actor visible in that prefix. Later moves in the same turn, and later reveals, do not enter it.*

Proof. At a `turn` event the importer builds the context from the prefix up to that boundary. When a later move arrives, it encodes the saved context; the current prefix is used only to reject ambiguous actors. No later prefix is substituted into a saved context. □

The selection of which examples to keep does use later identity checks. The proposition is about feature content, not selection independence.

The scout is an MLP: $384 \to 96$ (tanh) $\to M$. For the live format $M = 938$ moves and the network has 127,946 parameters. It minimizes

$$\mathcal{L}_{\text{scout}}(\psi) = -\frac{1}{N}\sum_{n} \log q_\psi(m_n \mid u_n) \tag{36}$$

with Adam at learning rate 10^{-3}, batches of 128, three epochs and gradient norm 1. Training uses at most 20,000 rows. That cap now binds. The rows used are the 20,000 with the smallest hash identifiers, a fixed pseudo-random tenth of the 192,835 M-C samples. A new sample enters training only if its identifier sorts inside the cap, so the subset behaves like a uniform subsample that refreshes slowly. Battles whose digest is 0 mod 5 form the held-out set, so no battle is split across partitions. A new scout is promoted only with at least eight held-out samples and no increase in held-out cross-entropy. At inference the softmax is restricted to the moves on a disclosed sheet, or to the moves historically seen for the species. The three most likely moves become hashed state features. Their probabilities are not renormalized, so their sum can be below one.

Proposition 13.2 (Cross-entropy decomposition)**.** *For a true label distribution p and a prediction q positive on p's support, $-\sum_m p(m)\log q(m) = H(p) + D_{\mathrm{KL}}(p\|q)$. The left side is minimized exactly at $q = p$.*

Proof. Add and subtract $\sum_m p \log p$. Then $D_{\mathrm{KL}} \geq 0$ by Jensen's inequality, with equality only at $q = p$. □

Caveat. The optimum concerns the distribution of *retained executed* moves, not opponents' intentions. Inference may also run mid-turn, for example at a forced replacement, whereas training contexts are taken at turn starts, so there is a possible distribution shift. Empirically, the scout lowered held-out cross-entropy, but in a matched battle comparison it changed no outcomes (0 improved, 0 worsened in 200 games), and recorded-state probes show the actor responds only weakly to scout features (`docs/offline-ppo-followup.md`).

13.2 Team versions as bandit arms

A team arm is an exact ordered list of sets together with its format. Its identifier is

$$v = \mathrm{SHA256}(\mathrm{canonicalJSON}(\{\mathrm{format}, \mathrm{sets}\}))_{1:20},$$

an 80-bit truncated fingerprint. Canonicalization sorts object keys, not the party list. Completed outcomes Y_i become scores $z_i = (Y_i + 1)/2$, and local and ladder evidence are queried separately. The posterior uses a $\mathrm{Beta}(1, 1)$ prior:

$$\alpha_v = 1 + \sum_i z_i, \quad \beta_v = 1 + \sum_i (1 - z_i), \quad \mu_v = \frac{\alpha_v}{\alpha_v + \beta_v}, \quad \sigma_v^2 = \frac{\alpha_v \beta_v}{(\alpha_v + \beta_v)^2(\alpha_v + \beta_v + 1)}. \tag{37}$$

Proposition 13.3 (Conjugacy, and the tie extension)**.** *Suppose binary outcomes are conditionally independent and stationary given p_v. Then w wins and l losses update* $\mathrm{Beta}(\alpha_0, \beta_0)$ *to* $\mathrm{Beta}(\alpha_0 + w, \beta_0 + l)$*. Fractional tie scores follow the same algebra under the pseudo-likelihood $\prod_i p_v^{z_i}(1-p_v)^{1-z_i}$, which is not a Bernoulli model of a three-outcome game.*

Proof. Multiply the density $\propto p^{\alpha_0 - 1}(1-p)^{\beta_0 - 1}$ by $p^w(1-p)^l$. Replacing the integer counts by score sums gives the same exponents, but a Bernoulli variable cannot take the value $\frac{1}{2}$. □

Example 13.4 (A played and an unplayed team)**.** *Seven wins and three losses give* $\mathrm{Beta}(8, 4)$*, $\mu = 2/3$ and $\sigma = \sqrt{32/1872} \approx 0.131$. An unplayed team has $\mu = 1/2$ and $\sigma = \sqrt{1/12} \approx 0.289$.*

With exploration enabled, the planner draws $\tilde{p}_v \sim \mathrm{Beta}(\alpha_v, \beta_v)$ independently and plays $\arg\max_{v \in C} \tilde{p}_v$. This is Thompson sampling [31].

Proposition 13.5 (Probability matching)**.** *Assume independent continuous posteriors over a fixed pool C. Then Thompson sampling selects arm v with probability equal to the posterior probability that v has the largest* true *success parameter: $\mathbb{P}(\text{select } v) = \mathbb{P}_{post}(p_v = \max_{u \in C} p_u)$.*

Proof. The sampled vector $(\tilde{p}_u)_{u \in C}$ has the same joint law as the true parameters $(p_u)_{u \in C}$ under the posterior. The event "v attains the maximum" is therefore equally likely for both vectors. Continuity makes ties have probability zero. □

Interpretation. The first edition stated this with "largest *sampled* parameter", which is circular: the algorithm selects that arm by definition. The content of the theorem is that sampled and true maxima share one law.

The estimate describes success *as piloted by the policy of the time, against the opponents of the time*. It is not intrinsic team quality, and the posterior is not conditioned on the actor revision. In production the team was pinned (one candidate, exploration off). Thompson selection was therefore inactive in the early campaigns, and every historical live result in Section 14 uses a single rain team. C13 later compared fixed teams through balanced live A/B rather than this Thompson sampler; its successful champion is evaluated in Section 16. Variants replace one member with an attributed same-species set and are validated in the simulator. Missing stat spreads are generated, labelled as such, and follow the move categories. A generated Garchomp spread that once boosted the wrong attacking stat was fixed for this reason.

Implementation: `ingest_public`, `public_lines` and `Scout` in `ml/scout.py`; `rank_teams`, `TeamPlanner.plan`, `suggest_variants` and `generated_spreads` in `ml/teams.py`; `fingerprint` in `ml/storage.py`.

14 Historical evidence: the actor and first search deployment

This section preserves the second edition's evidence as of 9 October 2026, recomputes the exact statistics, and states what each one does and does not show. Paired tests use the exact one-sided sign test of Section 10; all p-values below were recomputed and match the validation documents. Unless marked otherwise, sources are the repository's `docs/*-validation.md`, `docs/opening-policy.md`, `docs/post-campaign-learning.md` and `docs/search-agent.md`. Later results supersede preliminary live impressions in this historical section; see Sections 15 and 16.

14.1 Paired candidate evaluations

Figure 12 shows the paired experiments on one scale: the candidate's win rate minus the incumbent's, on the same seeds. Different rows use different panels, opponents and incumbents, so their totals are not comparable. Only the direction and the uncertainty of each row are.

Three patterns stand out.

- **Inference-time changes passed; learned weights mostly did not.** The large, independently confirmed gains came from the turn temperature (same weights, $\tau = 0.25$: 95 → 129 of 200) and from the strategic prior (125 vs. 106 of 160 on an untouched final). These are changes to how the network's scores are used, not to its learned residual.
- **Development wins regress.** `matchup-v1` (development $p = 0.0068$, final tie) and `strategic-v4` ($p = 0.0082$, then +1.5 points on fresh seeds) are textbook winner's curses (Section 10.3). Strata of the v4 confirmation show the tactical-opponent group *regressing*, from 45 to 38 wins of 55.
- **Search is the largest single effect.** The search agent's paired improvement, +24 points, is larger than any learned candidate's, with an interval that excludes zero.

14.2 The search agent offline

The opponent in the paired test is the search agent itself, piloting teams that real opponents brought, so the comparison is search against search versus policy against search. Two incumbent-arm games

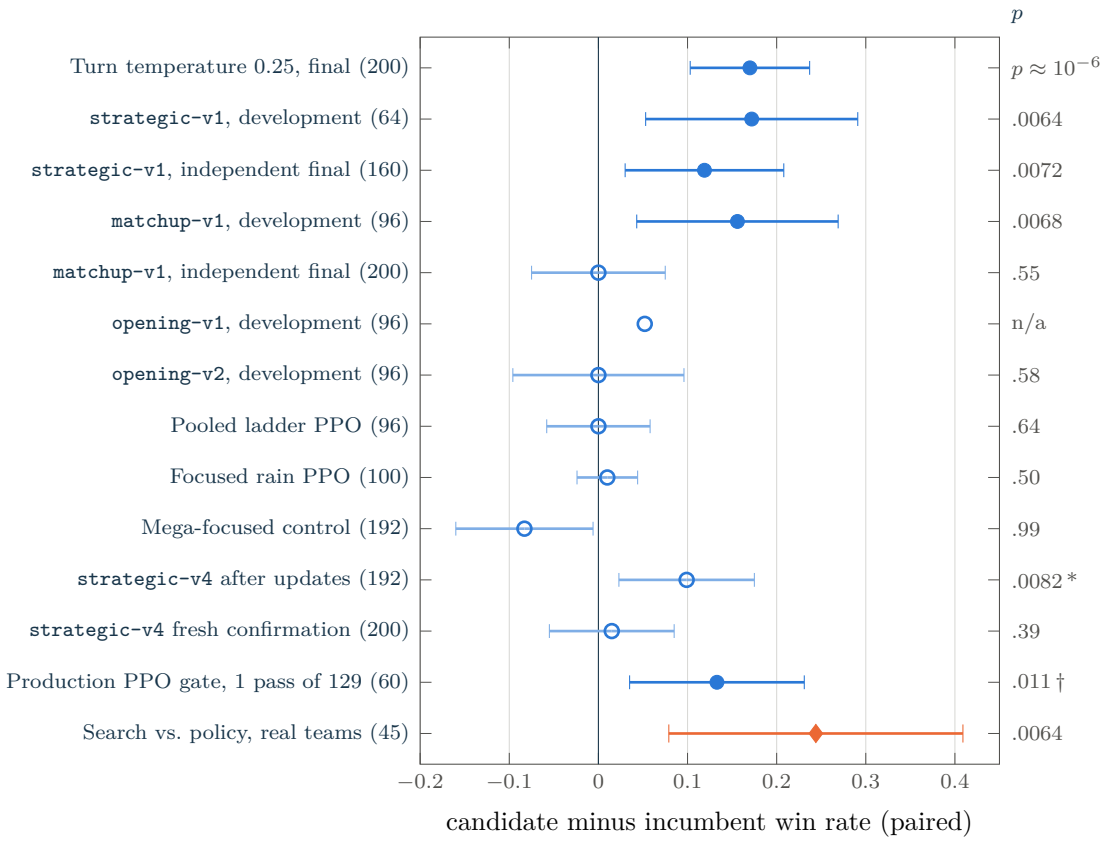


Figure 12: Paired evaluations. Filled markers passed their gate and hollow markers failed. The orange diamond is the search agent, compared with the learned policy on the same 45 seeds. Bars are approximate 95% intervals for a paired difference, $\pm 1.96\sqrt{[(g+\ell)/n - ((g-\ell)/n)^2]/n}$. The `opening-v1` panel did not report its discordant counts. * Met $p \leq 0.05$ but missed the then-required 10-point minimum by one win. † One pass among 129 gates; see Proposition 10.2.

Test	Result
Rain mirror, head to head, 40 games, sides alternated	Search 26–14 (65%); exact binomial one-sided $p = 0.040$.
Real teams, paired: same 45 seeds, sides and opponent teams; the opponent is search-piloted	Policy 19/45 (42%), search 30/45 (67%); 14 gains, 3 losses; McNemar one-sided $p = 0.0064$ (two-sided 0.013).
Same, clustered by opponent team (23 teams, each from both sides, one once)	13 teams favour search, 2 the policy, 8 tie; sign test $p = 0.0037$; exact sign-flip permutation $p = 0.012$.
Calibration: policy vs. search-piloted real teams	69/140 (49.3%), Wilson 95% [41.1%, 57.5%]; close to the live 41%.
Synthetic usage-sampled teams	Policy wins 9–1: *not* a valid proxy for the ladder.

Table 8: Offline search evidence, with open team sheets off. The run ledgers were recomputed for this report. `docs/search-agent.md` describes the paired panel as "45 real human teams"; it contains 45 games over 23 distinct teams. The team-level re-test shows that the effect survives this clustering.

had a degraded opponent (12 fallbacks from a forme lookup error, Appendix C). They were losses in both arms, so they do not change the discordant counts. The search configuration used for these runs was not recorded in the run files, which limits exact reproduction.

14.3 Why raw ladder win rate is not a strength measure

Showdown's ladder uses an Elo rating with a floor of 1000 and a rating-dependent K [17]. A player of rating R facing R_o has expected score $E = 1/(1 + 10^{(R_o - R)/400})$ [3]. A win adds $K_w(1 - E)$ and a loss subtracts $K_\ell E$. Above 1300, $K_w = K_\ell = 40$; from 1100 to 1299 both are 50. At 1000 the winner gains with $K_w = 80$ and the loser loses with $K_\ell = 20$, and these interpolate linearly up to 1100.

Proposition 14.1 (Stationary win rate under asymmetric K)**.** *Suppose a player keeps a constant true win probability p against opponents with expected score E, so that its expected rating drift is zero. Then*

$$p = \frac{K_\ell E}{K_w(1 - E) + K_\ell E}.$$

For evenly matched opponents ($E = \frac{1}{2}$), this is $K_\ell/(K_w + K_\ell)$: 0.20 just above the floor, about 0.35 near 1050, and 0.50 from 1100 upward. The formula assumes no clamp, $R - K_\ell E \geq 1000$. At the floor itself a loss costs nothing, so the stationary p is lower still.

Proof. The expected change is $pK_w(1 - E) - (1 - p)K_\ell E$. Setting it to zero and solving for p gives the formula. Substituting $E = \frac{1}{2}$ gives the special case. □

Interpretation. A player who is stable in the 1000–1100 band wins at most 50%, and as little as 20% or less near the floor, simply because of the asymmetric K and the clamp. The 847-game campaign's 41.1% is consistent with a policy that hovered in that band. A live goal of "50% win rate" is therefore partly a statement about the rating band. A steadily rising rating above 1100 is the more informative signal. GXE or a fixed-rating evaluation pool would be better still.

14.4 Live record

Games	Decision system	Record	Win rate
1–100	Legacy actor; temperature promoted before game 69	33–67	33%
101–375	Turn-temperature actor (overnight campaign)	113–162	41%
376–500	Same actor; corrected rain team	50–75	40%
433–532	Last 100 before `strategic-v1`	36–64	36%
533–632	`strategic-v1`, first 100	49–51	49%
1–847	All three generations	348–499	41% [38, 44]
41 games, 9 Oct	Search turns; greedy actor preview	27–14	66% [51, 78]

Table 9: Live ladder records with Wilson 95% intervals. These periods are not randomized comparisons. Team, rating band, opponent pool and transport all changed between them. The search row counts each battle room once (as of 20:07 UTC). It includes two losses in which no search decision was submitted, because of transport or team-validation incidents. Two battles abandoned to the timer during earlier transport crashes are not in this ledger; counting them gives 27–16.

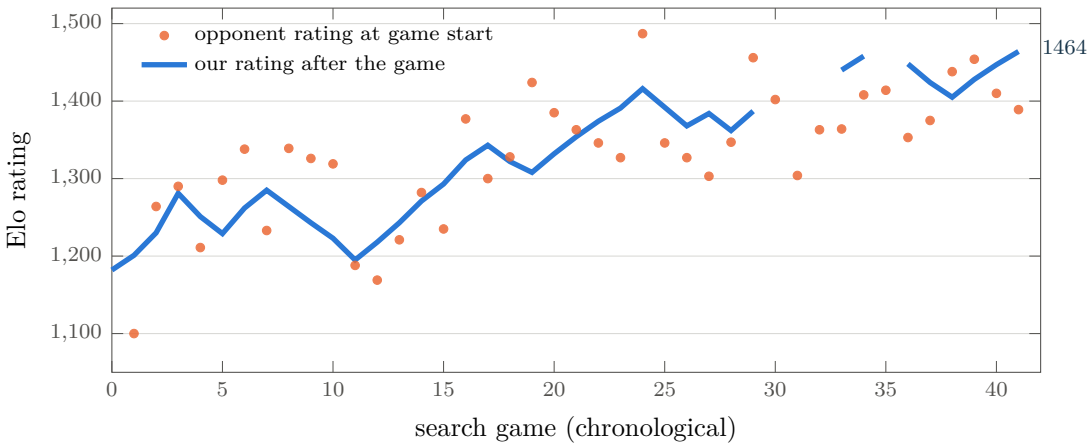


Figure 13: Live rating under the search agent on 9 October 2026. Gaps are games whose terminal log lacked our rating line. The climb from 1182 to 1464 took place in the symmetric-K region, where Proposition 14.1 predicts no drift for a player of stable strength.

Testing the live search record against Elo. Over the 37 search games whose logs give both players' pre-game ratings, Elo predicts $\sum_i E_i = 18.4$ wins, with variance $\sum_i E_i(1 - E_i) = 8.97$. We observed 23 wins, $z = 1.53$. The exact Poisson–binomial upper tail is $\mathbb{P}(X \geq 23) = 0.086$. Taken alone, the live sample is suggestive but not significant at 0.05. With the rating climb and the offline paired result, it is consistent with a real gain. It is *not* an independent confirmation of the offline effect size. A pre-specified live sample of a few hundred games, or rating trajectories compared over equal game counts, would settle it.

14.5 Offline PPO hyperparameter matrix

Seven settings (learning rate, entropy, collection size, minibatch, epochs and entropy decay) were trained with three seeds and five updates each. Each was evaluated against random and tactical opponents, with a sign test, a conditional bootstrap, and Holm correction over 14 development and 12 final-versus-control hypotheses [7]. No arm passed after correction. Two features of the design limit what it shows. The initial heuristic policy was already strong against random play, and arms with larger collections also took far fewer optimizer steps, which confounds data size with optimization exposure. The production hyperparameters were retained (`docs/offline-ppo-followup.md`).

14.6 Negative results and defects found along the way

An honest record includes what went wrong. The following were found and fixed, or are still open, during 7–9 October:

- **Label errors.** Five early wins were stored as losses after an identity change (corrected). A coloured public HP string parsed as full HP, which is still live in the deployed profile's state encoder (Section 3). Weather Ball was encoded as a 50-power Normal move in rain. A generated Garchomp spread boosted the wrong attacking stat.
- **Design errors.** Simulator seeds were truncated to 16 bits. The opponent schedule was confounded with team index. Fourteen orphaned scout labels were found. A score cap flattened 288 of 360 preview choices. Postgame feedback was silently dropped by the immutable-write rule.

- **Behavioural limits.** Preview entropy was 97.9% of uniform. The preferred four changed in only 4 of 100 cases when the opponent's six changed. The pressure curriculum never exercised its intended setup move.
- **Search defects** (Appendix C): a both-Mega enumeration drop, a forme lookup failure, unseeded sleep and confusion durations, and then-absent common random numbers. The appendix distinguishes repairs from remaining limitations at the third-edition cutoff.

15 Fourteen campaigns: from prediction to playing strength

The campaign ledger extends the early evidence through 31 October. A campaign is a fixed research question and protocol, not necessarily a model training run. Some fit networks, some change simulator search, and some only compare fixed alternatives on the ladder. Table 10 records the outcomes; the following subsections explain the mechanisms and the lessons. The notation g/ℓ denotes discordant gains and losses, never the candidate's ordinary win–loss record.

Table 10: Campaign ledger. C11 names the reconstruction and preview-development work, not a successful strength gate. C14 is ongoing.

Campaign	Fixed question / candidate	Recorded outcome
C1	Learned leaf values from live outcomes and self-play	No promotion. Six earlier development gates totalled 58/71; final candidate improved held-out log loss but its interrupted gates were inconclusive.
C2	Selective depth two and sibling action rankings	Depth: 53/43 over 394 completed pairs, 6 void, $p = .1792$; no confirmation. No clear superiority among 61 stored value nets on sibling rankings.
C3	Learned human opponent inside search	Development 2/9 at 70 pairs, futility; confirmation ledger 11/10 at 73, stopped. Rejected.
C4	Human preview imitation; matchup-value preview	Imitation 86/61 then 5/7, rejected. Value 81/52 then 87/62, both significant; deployed.
C5	Sigmoid leaf, selective Protect continuation, speed inference	Full bundle 44/23 dev, 52/38 confirm, 29/17 standard. Confirm missed; promoted by a documented post-results amendment, with a live rollback rule.
C6	Twelve worlds; wider 36-by-32 screen	42/39 and 37/38 respectively; neither passed. Existing compute budget retained.
C7	Evaluation/depth/bench bundle; larger preview ensemble	PLAY 45/52, PREVIEW 61/88; neither promoted.
C8	Public-history scouting with simulated coverage	17/14, insufficient for promotion.
C9	Live 2^4 factorial: Protect, boosts, bench, scouting	All four factors H0 under their normal-approximation sequential tests. Final boost decision at 1,581 randomized games, estimated +1.3 Elo.
C10	Bounded preview guard; learned screening plus endgame depth	Preview 8/10, futility at 400 pairs. Combined search 108/106 over 785 completed pairs, 15 void, $p = .47276$. Neither qualified for live confirmation.
C11	Illusion/Recharge repairs; MPS preview residual	Correctness repairs shipped; residual failed predictive holdout, so no strength gate.
C12	Shared belief-game root policy and bounded exploitation	46/59, 400 pairs, $p = .91422$, futility. No live test.

Campaign	Fixed question / candidate	Recorded outcome
C13	Two tournament-derived teams under the same pilot	Helm H0 at 302 games; Mev H1 at 897, +61.9 Elo estimate. Mev becomes champion.
C14	Human-imitation preview versus pvalue on Mev	Active; 9 completed games at the frozen cutoff. No strength conclusion.

15.1 C1–C2: outcome prediction is not counterfactual ranking

The first search-learning loop finally trained a component used by turn search rather than a sampled actor used only in a separate self-play system. Completed live ledgers were reconstructed into positions and labelled with verified results; CPU self-play added omniscient examples. The MPS trainer split by game, calibrated on older live games and reserved the newest 40% as a test. Five-seed ensembles reduced fitting variance and were distilled into an engine-readable network.

The strongest reported C1 candidate improved live-test log loss from .538 to .497 and AUC from .793 to .831. Yet its development 12/11 and confirmation 9/5 ledgers were stopped without a pass. Earlier gates aggregated 58/71. The live policy therefore stayed on the hand evaluation through the 1,018-game review. Fitting who eventually won does not identify which alternative action would have caused a better outcome.

Calibration put learned values on the hand evaluation's scale. If $\mathbb{P}(\text{win} \mid h) = \sigma(ah+b)$ and the network emits logit z with live temperature T, the converted score was $u = (z/T - b)/a$, clipped to $[-20, 20]$, and the blend was $(1-\beta)h + \beta u$. This avoids silently changing the units of opponent softmax temperatures or missing-world penalties. It does not remove the observational nature of the labels.

C2 addressed ranking directly. Sibling groups held the reply and chance stream fixed while changing one side's action and playing out the remainder. Of 6,400 groups, 2,629 had unequal outcomes and informed the metric. Hand pair accuracy was .6498; the best of 61 nets reached .6553, difference .0055 with a group-bootstrap interval $[-.0029, .0134]$, without adjustment for selecting that net. The metric agreed with only 8 of 15 historical gate orderings (Spearman $-.029$). It was not validated as a promotion shortcut. Selective second-turn search separately failed at 53/43, despite 47.48% more simulations per decision. Pseudo-move errors voided six pairs; even counting all six as gains would not have passed the fixed test.

Implementation: `docs/value-learning.md`; `search/train_value.py`, `search/sibling_metric.py`, `search/ledger_positions.py`, `search/selfplay.cjs`, `search/value_gate.py`.

15.2 C3–C4: learn what ladder opponents actually do

The original gate opponent was search piloting real teams. Real team sets did not make its decisions human. Public replays supported a separate imitation opponent and new preview models. About 150,000 replay games were collected; the first move and preview fits used about 10,600 and 12,600 respectively. Later analyses used filtered, rated subsets rather than a single universal corpus count.

For a legal joint-action set $\mathcal{A}$, the move model assigns logits $f_\psi(h, a)$. An executed public log can leave one slot's intended move unknown after a flinch or knockout. With consistent label set $C \subseteq \mathcal{A}$, the loss is

$$-\log \sum_{a \in C} \frac{e^{f_\psi(h,a)}}{\sum_{u \in \mathcal{A}} e^{f_\psi(h,u)}}. \tag{38}$$

About 19% of labels needed this marginalization. Inputs combine 146 position features, 70 joint-action features and move/species embeddings. Whole games stay within one split. Reveals later in

the replay can recover facts about the acting player's own sets for retrospective training; they are not facts available to our live opponent model at that earlier turn.

The preview imitation is listwise over 90 plans: $\binom{6}{2}\binom{4}{2} = 90$, treating both leads and bench as unordered pairs. The transport maps the selected plan back into a legal ordered preview. This differs from the actor's 180 ordered-lead classes in Proposition 4.2. A separate matchup-value classifier learns $P(\text{win} \mid \text{own six}, \text{opposing six}, \text{plan})$ from human outcomes; it is an observational predictor, not an identified causal value for all unplayed plans.

On 957 identifiable live opponent actions, the deployed equilibrium/greedy model achieved top-1 .176 and NLL 3.70; the .8-human/.2-deployed mixture achieved .264 and 2.82. C3 nevertheless failed its gate. C4's imitation preview passed development and failed fresh confirmation. The value preview passed both (81/52, then 87/62) and was deployed on 14 October. Its early live segment was 1236 ± 28 over 635 games versus the preceding 1206 ± 19 over 1,325. That chronological comparison suggests a modest improvement but is not randomized evidence for the preview alone.

Implementation: `docs/human-opponent.md`; `search/human/extract.py`, `search/human/train_policy.py`, `search/human/preview_model.py`, `search/human/preview_value.py`, `search/human/imitation.py`.

15.3 C5–C8: risk attitude, horizon and an imperfect gate

C5 diagnosed a horizon effect: Protect preserves material for a leaf even when the next turn remains lost. A live logistic calibration over 12,896 positions gave $P(\text{win}) = \sigma(.801h - .430)$. The deployed leaf became

$$L(h) = \frac{4}{.801}\,\sigma(.801h - .430). \tag{39}$$

Although monotone in h, the transformation changes decisions when values are averaged over stochastic outcomes. Its convex region below the sigmoid midpoint can favour an uncertain escape over certain slow defeat; its concave region above it can favour preserving a win. The empirical calibration is not a guarantee that every counterfactual leaf is a true win probability. A second turn re-evaluates the best Protect and non-Protect lines when the world's material value is below -1. Speed-order observations constrain usage draws, with exclusions for priority ambiguity and mid-turn Speed changes. The equilibrium weight was changed from .5 to .25 as part of the bundle.

The amendment must remain visible. Leaf-only failed (36/32). The full bundle passed development (44/23, $p \approx .007$), missed confirmation (52/38, $p \approx .085$), and showed no significant harm against standard search (29/17). The original automatic promoter declined. After the stages finished, amendment 1 pooled confirmation with the different standard-opponent stage (81/55, nominal $p \approx .016$) and promoted anyway. This was a deliberate post-results exception, not a pass of the original preregistered rule. A prospective live rule required at least 300 games and rollback below the C4 performance segment. C5 cleared it; the eventual segment was 1287 ± 21 over 1,169 games. A room-scoping browser repair shipped in the same deployment, so the segment cannot isolate the search bundle's effect.

C6 enlarged worlds and screening separately; neither passed. This rejects those fixed candidates under their gate, not the general importance of sampling noise. C7 combined boost/Speed/evasion/weather valuation, broader Protect continuation, endgame depth, Mega avoidance and a corpus back-pair prior. PLAY failed 45/52; the larger preview ensemble failed 61/88. C8's public-history scouting achieved high item precision in diagnostics but only 17/14 in its gate. None of those options became the default.

The gate itself was then audited. Its argmax imitation Protected on 29% of actions versus about 12% for ladder humans, switched about 2.8% versus 8.4%, and lost to our search roughly three quarters of the time. Requiring two significant stages with repeated early futility made modest gains hard to detect. Better predictions and changed move frequencies were useful candidate-development evidence, but inadequate substitutes for target-domain wins. C9 moved the experiment onto the ladder.

Implementation: `docs/endgame-search.md`, `docs/thinking-budget.md`, `docs/c7-live-review.md`, `docs/c9-live-ab.md`; `search/speed.py`, `search/scout.py`, `search/config.py`.

15.4 C9–C11: direct experimentation and focused candidates

C9 randomized four binary factors in balanced 16-game blocks. Wider Protect continuation, boost-/Speed/evasion terms, a back-rate bench prior and online scouting each had a 0-versus-+40-Elo sequential comparison. Every factor stopped at H0 (Table 11). The early boost estimate of +19 Elo at 1,223 games shrank to +1.3 at its 1,581-game stopping decision. H0 here chooses zero over the specified +40 alternative; it does not establish exactly zero effect, harmfulness or equivalence. The completed campaign ledger contains 1,582 games because a result arrived after the stopping snapshot. Decisions must use the saved first decision, not a later summary.

C9 factor	Randomized games	Score on / off	Elo estimate	LLR
Protect	431	.4769 / .5395	−43.8	−2.252
Boosts	1,581	.5006 / .4987	+1.3	−2.423
Bench	397	.4750 / .5533	−54.9	−2.429
Scouting	752	.4880 / .5199	−22.2	−2.606

Table 11: C9 immutable stopping receipts. All crossed the H0 boundary −2.251. Main effects use only the games while that factor was randomized; frozen later games do not enlarge its sample. Estimates are descriptive.

C10 froze two narrower candidates and used a pooled 800-pair offline rule, with only rejection looks at 400 and 600 and one final significance test. The preview guard penalized Intimidate leads in selected opposing matchups only among plans within .025 of the incumbent's top probability. It stopped at 8/10 over 400 pairs. The combined screening/endgame candidate trained on 1,500 baseline games (1,186,801 action rows); MPS fitting took 8.1 seconds. Its held-out final-choice retention was 99.11% and estimated simulation saving 15.12%. Those engineering metrics qualified the fixed bundle for a strength test, which ended 108/106, $p = .47276$, over 785 completed and 15 void pairs. Neither candidate reached the required fresh live test.

C11 repaired Illusion and Recharge reconstruction and explored a bounded preview-logit residual. Its 8,868 examples were split chronologically 70/15/15 by whole game. A 32-unit residual, capped at ±.35 logits, trained in 1.53 seconds on MPS. Validation improvement .001055 missed the required .002, and test log loss worsened from .647704 to .648898. The candidate remained disabled rather than tuning against the failed test set. Repairs and scheduling improvements are discussed in Section 17; they are not claimed as measured Elo gains.

Implementation: `docs/c9-progress-review.md`, `docs/c10-m5-campaign.md`, `docs/c11-reconstruction-and-preview.md`; `search/ab_test.py`, `search/campaign.py`, `search/train_preview_residual.py`.

15.5 C12: a shared Bayesian root strategy

The baseline commits to one root action, but it solves hidden worlds separately and averages an action only across screens that retained it. This is selection-dependent coverage: an action worth 1 in its only retained world and 0 in five omitted worlds receives $1 - .5 = .5$, beating an action worth .4 everywhere, although its full-world mean is only 1/6. Replacing this rule requires completing comparable rows, not filling missing payoffs with guessed constants.

C12 nominated up to 24 actions, completed each against every retained reply in every successful world, and reused the original serialized states and chance streams. For posterior weights p_w and complete matrices M_w, it solved the one-sided private-type game

$$V^* = \max_{x \in \Delta(\mathcal{A})} V(x), \qquad V(x) = \sum_w p_w \min_{b \in \mathcal{B}_w} \sum_a x_a M_w[a, b]. \tag{40}$$

One x commits our joint action across worlds; the opponent may respond to its own private type. Averaging matrices before taking the minimum would erase that information advantage. The implementation uses equal weights, nonnegative cumulative regrets, linearly weighted averages, and a recorded primal–dual gap

$$G = \max_a \sum_w p_w (M_w \bar{y}_w)_a - \sum_w p_w \min_b (\bar{x}^\top M_w)_b. \tag{41}$$

Proposition 15.1 (Scope of a shared-root certificate)**.** *For any complete finite matrices and probability vectors $\bar{x}, \bar{y}_w$, $V(\bar{x}) \le V^* \le U$, where $U = \max_a \sum_w p_w (M_w \bar{y}_w)_a$. Thus $G = U - V(\bar{x})$ bounds the suboptimality of $\bar{x}$ in this sampled game.*

Proof. The first inequality follows by maximization over x. For any x, each minimum over replies is at most the expectation under $\bar{y}_w$, so $V(x) \le \sum_w p_w x^\top M_w \bar{y}_w \le U$. Maximize over x. □

Bounded exploitation moved along the line from $\bar{x}$ toward a pure best response to the existing opponent prior, choosing the largest movement with $V(x) \ge V(\bar{x}) - .1$. Concavity of V makes the feasible portion of this line an interval, permitting bisection. This is a restricted line search, not the full constrained optimal-exploitation solution. The principle of trading model exploitation against worst-case robustness has antecedents in restricted Nash responses [8]. A separate RNG sampled the entire joint action, retained across submission retries. Existing selective deeper estimates replaced their actual matrix cells before the shared solve; other cells remained one-turn leaves. This fixed bundle changed both coverage and the old scalar correction, so it was not an isolated test of randomization.

On 48 frozen positions the diagnostic gaps were all below .01 (median .00798), and the extra simulations were 13.4%. Twelve fresh adaptive contexts had legal choices and no root fallback; median latency rose from 5.18 to 6.26 seconds. These were engineering checks. The frozen strength panel rotated incumbent search, argmax imitation and sampled imitation, using both sides. It stopped at 400 pairs with 46/59, $p = .91422$, 14.29% more simulations per decision, and no live confirmation.

The negative result does not contradict the certificate. The guarantee concerns sampled types, retained replies and approximate leaf payoffs. Opponent sets are imperfect, the opponent is effectively informed about our set, and deeper continuations can still depend on determinized hidden information. Depth-limited imperfect-information solving generally needs richer continuation constraints than one scalar full-state value [1]. C12 improved the definition and solution of the surrogate root game without establishing stronger play in the target game.

Implementation: `docs/game-theory-research.md`, `docs/competitive-review-20261024.md`, `docs/c12-belief-game.md`; `search/belief.cjs`, `search/agent.py`, `search/engine.cjs`.

15.6 C13–C14: change the team, then test its preview

C13 reconsidered the fixed team after search campaigns had not broken the plateau. Human corpus estimates were weighted equally by player to avoid letting prolific strong accounts dominate team rankings. The 111,564 rated games / 197,708 eligible sides showed wide team-effect intervals, not a proven universally superior six. Live matchup audits suggested weaknesses against Fairy attackers and Competitive/Defiant teams. Those exploratory patterns motivated candidates; their estimated deficits were not added up as a forecast of a team's improvement.

The tournament-derived Helm rain candidate retained much of the Recife core, adding Gengar and Rillaboom; Mev combined Pelipper rain and Charizard-Y sun with Archaludon, Grimmsnarl, Venusaur and Garchomp. Imported moves, items, abilities and natures followed the attributed sheets; hidden stat points used modal usage spreads. This distinction matters: an attributed team archetype is not a claim to know the tournament player's exact hidden stats. Both candidates passed simulator validation. The randomized king-of-the-hill comparison is reported in Section 16.

After Mev won, its preview still used Archaludon and Grimmsnarl in every recorded plan, Charizard in 95%, Pelipper in 87%, Garchomp in 16% and Venusaur in 3% of the 885-game diagnostic. The same four occurred in 83%. In 1,046 public games from 298 players with the exact six, Venusaur was seen in 42% and Garchomp in 53%. Public appearance is a lower bound on being brought; it is not an exact counterpart of the bot's private preview record. Nevertheless, Venusaur's absence identifies a plausible unused sun mode.

C14 fixes Mev and the turn search, comparing pvalue with `preview-1455f4a55d18112a:argmax`. On the same 885 opposing previews, that imitation would bring Venusaur in 63% and lead Charizard–Venusaur in 558 games. This is a counterfactual plan-frequency diagnostic, not a counterfactual win estimate. The experiment began on 31 October with the same 0/+40-Elo, 300–3,000-game rule. At the paper cutoff only nine games had finished: no decision is permitted, and no improvement is claimed.

Implementation: `docs/c13-team-ladder.md`, `docs/c14-preview.md`; `search/human/team_effects.py`, `scripts/team_ladder.py`, `evidence-20261031.json`.

16 Live experimentation and the measured breakthrough

16.1 Three scales that must not be conflated

The account's latest Elo is a stateful matchmaking variable. A deployment's performance rating estimates a constant strength against the opponents actually faced. A randomized arm contrast estimates the effect of changing a component while the account, time period and surrounding software are shared. They answer different questions. Figure 14 displays performance segments; Table 12 gives the controlled contrast.

For outcomes $s_i \in \{0, \frac{1}{2}, 1\}$ and known pre-game opponent ratings O_i, `search/elo_monitor.py` solves

$$\sum_i \frac{1}{1 + 10^{(O_i - \widehat{R})/400}} = \sum_i s_i. \tag{42}$$

Under independent Bernoulli outcomes and a fixed Elo logistic model, let $c = \log(10)/400$ and $p_i = 1/(1+10^{(O_i-\widehat{R})/400})$. The log-likelihood score is $c\sum_i(s_i - p_i)$ and the information is $c^2\sum_i p_i(1-p_i)$. The implementation's approximate interval half-width is therefore

$$1.96\,\frac{400/\log(10)}{\sqrt{\sum_i p_i(1-p_i)}}. \tag{43}$$

Tie scores use the same estimating equation rather than a separately fitted draw model. All-wins/all-losses samples clamp to the supported range and have no finite ordinary interval. Opponent skill drift, repeated opponents and serial dependence make these intervals descriptive. They are neither sequential confidence sequences nor independent replications of a prior gate.

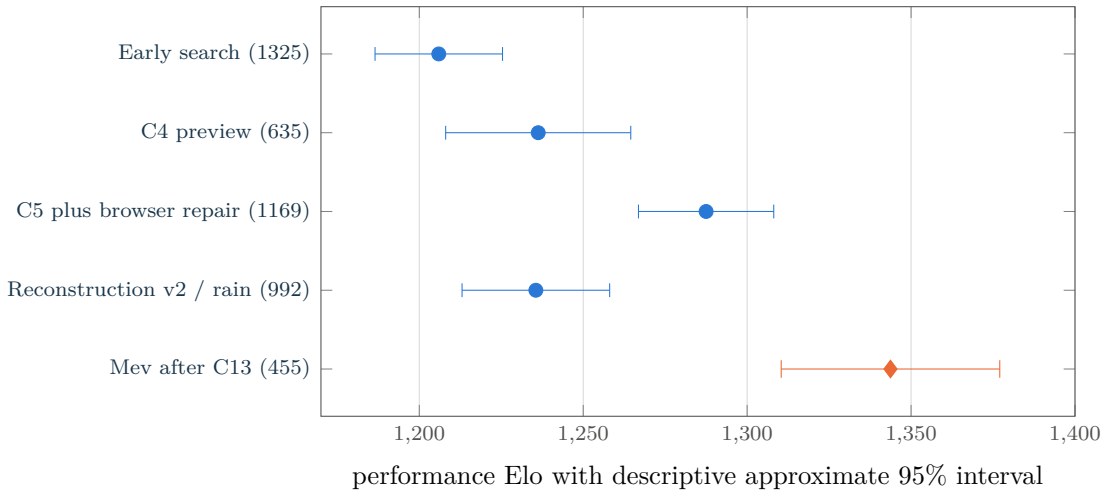


Figure 14: Chronological performance cohorts, with sample size in parentheses. The Mev cohort joins all completed C13final assignments across the permanent team-setting restart. The earlier rows are contiguous non-A/B policy segments 6, 7, 8 and 13 in the frozen monitor snapshot. Time, opponents, repairs and policy differ across rows: this chart is not a causal decomposition.

16.2 Balanced live assignment and the sequential rule

C9–C14 use the same interleaved A/B infrastructure. A frozen JSON spec names base settings and binary factor overrides. With k unfrozen factors, a seeded permutation of all 2^k combinations fills each balanced block. Assignment is stored by room before play; a resumed room reuses its arm. A one-factor comparison alternates the two treatments in seeded two-game blocks. Both arms share one account and matchmaking history. This makes contemporaneous opponent populations comparable in expectation, without claiming that each arm faced identical human opponents or that time and rating feedback disappear entirely.

Let $\bar{s}_1, \bar{s}_0$ be on/off scores, $d = \bar{s}_1 - \bar{s}_0$, and S^2 the pooled sample variance across all on/off game scores. The code uses $V = S^2(1/n_1 + 1/n_0)$ and the transform

$$d_j = \frac{1}{1 + 10^{-E_j/400}} - \frac{1}{2}, \qquad \Lambda = \frac{d_1 - d_0}{V}\left(d - \frac{d_0 + d_1}{2}\right), \quad (E_0, E_1) = (0, 40). \tag{44}$$

The nominal Wald boundaries [32] are

$$\log\frac{\beta}{1-\alpha} = -2.2513, \qquad \log\frac{1-\beta}{\alpha} = 2.8904, \quad \alpha = .05,\ \beta = .10. \tag{45}$$

No decision is allowed below 300 completed games. An upper crossing accepts H1, a lower crossing H0; a 3,000-game cap is inconclusive and keeps the incumbent. The implemented test is a normal approximation with estimated variance, minimum sample and cap. It does not inherit exact anytime error control merely by using Wald's boundaries. Dependence, factorial interactions, multiple factors and sequential candidate selection further qualify the nominal error interpretation. Four C9

factors at nominal .05 each were not a family-wise .05 experiment; neither were C13's two successive team tests.

The displayed Elo effect is

$$\widehat{E} = 400 \log_{10} \frac{.5 + d}{.5 - d}. \tag{46}$$

It maps a score difference to an Elo advantage against an equal baseline. It is not exactly the difference between two independently fitted performance ratings, although they are close for C13. A fixed-sample interval obtained by transforming $d \pm 1.96\sqrt{V}$ is descriptive after optional stopping. No such interval replaces the actual stopping rule.

Outcome and missingness policy. Forfeits and opponent timer losses are genuine terminal wins and count. Excluding them would condition on a consequence of good positions as well as on opponent behaviour. An assigned room without a terminal record is excluded until it finishes; it is not scored as a tie. An outdated module docstring says unfinished games count .5, but the actual `join` consumes only completed terminal games. The frozen article audit follows that code. A future audit should continue reporting missing assignments by arm, because terminal-only analysis can still be biased by treatment-dependent outages. Real ties count .5; none occurred in the frozen 7,734-game live dataset.

16.3 C13: the team comparison that passed

The incumbent was `Rain-Recife-special-stat-fix`: Garchomp, Politoed, Incineroar, Archaludon, Farigiraf and Golisopod. Round one tried `Rain-Gengar-Helm-Baltimore`; round two tried `SunRain-Mev-Brisbane` against the surviving rain champion. Both arms used the same C5 search and C4 pvalue preview. Only the exact six sets changed.

C13 arm	Games	Wins	Score	Performance	Mean opponent
R1 Helm	151	66	.4371	1132 ± 57	1178.9
R1 Recife	151	82	.5430	1204 ± 57	1172.6
R2 Mev	448	249	.5558	1286 ± 34	1243.5
R2 Recife	449	210	.4677	1225 ± 34	1249.6

Table 12: First-stopping C13 cohorts, recomputed by joining completed games to saved room assignments. Later arrivals remain in the audit ledger but do not change the 302- and 897-game decisions. Intervals are approximate descriptive 95% performance intervals, not sequential arm-effect intervals.

Helm stopped H0 at 302 games: $d = -.10596$, estimated -74.8 Elo, $\Lambda = -2.323$. Mev stopped H1 at 897 games:

$$d = 249/448 - 210/449 = .0880976,$$
$$\sqrt{V} \approx .0334, \qquad \widehat{E} = 61.9, \qquad \Lambda = 3.054 > 2.890.$$

The independent performance-rating difference is 61.5, close to the score-transform estimate. Mean opposing ratings differed by only 6.1 points, with the challenger facing slightly lower opposition. The direct randomized contrast is stronger evidence than comparing two peaks or entire deployment periods. It supports the Mev team under this fixed preview and turn pilot; it does not establish universal team quality or isolate any one species. The effect estimate was selected at a successful stopping time and may regress; further independent ladder evidence remains valuable.

After the decision on 30 October, `c13final` played Mev every game. The frozen cohort contains 455 completed games, 226 wins and 229 losses, performance 1343.7 ± 33.3, against mean opposition 1346.9. Its descriptive lower interval endpoint is 1310.4. The 49.7% win rate is compatible with improvement because matchmaking supplied stronger opponents. This cohort corroborates sustained play above the old plateau, but is a subsequent observational cohort rather than another randomized treatment comparison. C14 games are excluded from it.

The runner's default team changed permanently on 31 October. Before that, run-level metadata still named the rain default while the per-game spec selected Mev. Accordingly, the article identifies the champion cohort from *assignment ledgers*, not by filtering the default team name. Two monitor segments across that restart are combined. This corrects a subtle measurement trap without changing any campaign result.

16.4 How the evidence changed the research process

The early gate measured sampled actor play; search deployment made that objective mismatched. The next gate used search opponents; imitation brought some human behaviours closer, but retained serious action-distribution and strength differences. C9 made target-domain interventions directly testable. C10 and C12 subsequently required an offline pass merely to qualify for a fresh live experiment, never as sufficient evidence for automatic live gain. C13's team-only contrast finally passed that direct ladder test.

The pooled offline rule also addressed statistical power and wasted compute. At 20% discordance, a 20,000-campaign simulation found a +4-point gain passed the intended old two-stage rule about 9.5% of the time versus 78.7% for the pooled 800-pair rule; null pass rates were .06% and 4.30%. These are assumptions-based power simulations, not observed policy gains. A better rule cannot fix a poor surrogate opponent, as C12's subsequent rejection and C13's direct test illustrate.

Implementation: `search/ab_test.py` (assignment, terminal join, score transform, normal-approximation SPRT); `search/elo_monitor.py`; `scripts/team_ladder.py`; `docs/m5-pipeline-review.md`; `evidence-20261031.json`, `collect_evidence.py`.

17 Software and compute: making experiments survive real play

The campaigns depended on software work that is distinct from strategic strength. A faithful reconstruction, a successful model fit and a reliable browser submitter each remove a failure mode; none alone demonstrates an Elo gain. This section records those changes and their measured scope.

17.1 Search traces, per-team data and live priority

The actor's SQLite episode is no longer the only data path. Search decisions retain the private request, choice, sampled-search trace, timing and configuration in append-only per-run JSONL. Completed games are reconstructed into labelled positions for value learning and grouped by exact team fingerprint. Per-team stores separate games, live positions, self-play, value candidates, screening labels, gate results and promoted pointers. This prevents a new team's examples from silently being treated as the previous team's distribution. Content checksums protect engine-readable model files; a missing or invalid pointer falls back to the hand evaluator or probe screen.

The value service can ingest in a live-only role while producers, training and promotion remain idle. Repeated new games therefore grow a dataset without necessarily changing the deployed policy. The old PPO supervisor is paused during the search campaigns. There is no claim that the current player updates its strategy after every match.

CPU contention was more consequential than GPU utilization. At ordinary background QoS, concurrent work raised median live inference from roughly 2.3 to 10 seconds. macOS background QoS restored live responsiveness but starved the simulations. The implemented alternative registers offline process groups and pauses them around live search with SIGSTOP/SIGCONT, letting them use ordinary scheduling while the human opponent thinks. A stale pause marker is recovered after a crashed player. Later compute-priority coordination also accounts for shared workstation workloads; it does not control unrelated user applications. Campaign resource overrides permitting RAM and swap do not make CPU interference free.

Implementation: `search/team_data.py`, `search/value_store.py`, `search/value_service.py`, `ml/cluster.py`; `docs/value-learning.md`, `docs/model-storage.md`.

17.2 MPS is faster at fitting, not at JavaScript simulation

A learned screen scores joint actions before the expensive probe and matrix simulations. It uses the 146-position plus 70-action inputs and a two-hidden- layer 128-unit MLP, trained on within-world standardized probe scores. The objective imitates the existing ranking, so low fitting loss is primarily an efficiency opportunity. Both game-grouped validation and a newest-30% chronological test constrain retained actions, final-choice survival and simulation savings. Test failures reject the fixed prefilter rather than retuning its size against the test.

Workload / diagnostic	Measurement and limit
M5 screening optimizer, batch 4096	CPU 1.230 ms/step, MPS .565 ms/step; 2.18× fitting throughput. Synchronized timing; not a whole-player speedup.
1,000-game screening pilot	777,435 action examples; 1.42 s, 7 epochs. Chronological final-choice retention 98.78%; estimated saving 15.10%.
Twelve matched held-out positions	78.15 to 60.29 s total search time (−22.9%); 67,174 to 54,890 simulations (−18.3%); 9/12 identical choices, no fallback. Small shared-host sample.
C10 fixed learned-screen/endgame bundle	1,186,801 action rows from 1,500 games; fit in 8.1 s; predictive checks passed but strength gate did not.
C11 six versus eight engines	Pause-adjusted .642 versus .653 positions/s: six within 1.7%, with 25% fewer workers. Not an isolated silicon maximum or full-game throughput comparison.

Table 13: Hardware and screening measurements. Faster fitting or fewer simulations is not evidence of more wins. The pinned simulator remains CPU JavaScript; no GPU simulator was implemented.

The original screening-only gate required point-estimate non-inferiority and at least 20% measured simulation saving. It was not an error-controlled non-inferiority test with a declared margin. C10's combined screen/endgame protocol instead required superiority in wins for its fixed bundle, after at least 15% predicted savings. This is a different preregistered question, not a retrospective claim that the earlier screening gate passed. Training takes seconds; evaluation takes hours or days. The GPU's idle time is therefore not evidence that more epochs would remove the strategy plateau.

Implementation: `search/screenfeat.cjs`, `search/train_screen.py`, `search/compute_bench.py`; `docs/learned-screening.md`, `docs/m5-pipeline-review.md`, `docs/c10-m5-campaign.md`, `docs/c11-reconstruction-and-preview.md`.

17.3 Mechanics and reconstruction repairs

An official simulator is exact about the state supplied to it. It cannot repair a tracker that supplies a different game. The following repairs therefore belong to the software history, including failures first listed in the second edition's appendix.

- **Enumeration and targeting.** Opposing double-Mega worlds now enumerate Mega and non-Mega variants before applying the one-Mega limit. Harmful ally-targeted status moves such as Parting Shot are pruned alongside damaging ally moves. Struggle rebuilds an exhausted moveset rather than feeding an impossible ordinary move into the simulator.
- **Identity and slots.** Ally Switch swaps the tracked actives. Illusion's `|replace|` is accepted without an HP field, transfers observations to the revealed identity, and restores previously known facts about the disguise. A switched-in healthy disguise no longer retains the illusion user's faint flag. Regression positions reproduced an all-worlds failure that formerly fell back to the first legal action.
- **Counters and pseudo-moves.** Failed Protect resets the chain. Public `-mustrecharge`, its consumption, and authoritative private Recharge requests restore the volatile; the targetless pseudo-move retains its request slot rather than undergoing ordinary name-based targeting.
- **Species and beliefs.** Forme aliases repair Floette-Eternal's Mega prior and own-set lookup. Equal-priority move order constrains sampled Speed, subject to explicit ambiguity exclusions.
- **Trace and chance integrity.** Preview/single-choice decisions clear the preceding trace. CRN matches chance streams across our matrix rows. C12 completion caches the original serialized world, including sampled sleep/confusion counters; regenerating missing cells from a new root would resample those counters and compare different games.

Sleep/confusion reconstruction still calls `Math.random` on the ordinary build path. C12 preserves the sampled counters inside its cached roots; this does not make all ordinary reconstructions reproducible from the exposed search seed. Opponent PP and some history-dependent state remain incomplete. Those residual limits are retained in Appendix C.

Implementation: `search/tracker.py`, `search/agent.py`, `search/engine.cjs`, `search/priors.py`, `search/speed.py`; commits `b12314c`, `4fb2e91`, `cc32075`, `58bc518`.

17.4 Transport reliability changes the experiment actually played

The answered-request cache originally omitted the room. Identical preview or opening requests in a new game could look answered. The player stalled until its fallback submitted the third-ranked preview. This affected 231 of 634 audited C4 games (36%); they won 47.6% versus 52.1% for the others, a descriptive association rather than a randomized bug-effect estimate. Commit `4b2800f` scoped the key to the room. That fix shipped with C5, which is why the later segment cannot assign the whole observed rise to search alone.

Team A/B added another transport state: which exact teams had already been imported into the browser session. Remembering every imported team avoids re-importing from an incompatible teambuilder view on the next arm switch. The per-game override is validated before use; unavailable teams or preview assets fall back to the baseline and emit an explicit `ab_error`. Such events must be audited as treatment-delivery failures rather than quietly presented as successful treatment games.

A browser page then froze during C13. All page evaluations hung, so the 180-second stall check in the same asynchronous loop never ran. The fix is an independent thread probing the page every 60 seconds; after 300 seconds without response it exits with code 75 and launchd restarts the player.

An account lock and active-room resume prevent a second controller from starting another account session. This bounds a class of hung-page failures; it does not recover every server disconnect or guarantee no timer loss.

The final team-setting handover also exposed a configuration-editing trap: array replacement in a plist inserted an extra argument, causing a brief startup failure. The corrected argument array was reloaded. Operational handover, browser caching and model selection are distinct layers; each can invalidate an otherwise correct strategic comparison if not logged.

Implementation: `scripts/browser_play.py`, `ml/browser.py`; commits `4b2800f`, `a459c01`, `f22fa2c`; `docs/c13-team-ladder.md`.

17.5 Pinned deployment and durable stopping decisions

The player runs from a pinned live worktree, and each offline campaign uses its own pinned source with frozen assets. Editing the development checkout therefore does not mutate a game in progress or a running gate. Deployment tests the proposed source, scans its public contents, selects the pinned revision and requests a graceful restart at a game boundary. Model and experiment changes are likewise adopted between games. Changing a launchd plist requires unloading and bootstrapping it; merely restarting a loaded job does not reload its configuration.

Persistent supervisors hash inputs, take locks, adopt existing worker PIDs or resume original ledgers, and retry interrupted games on their original seeds. Three exhausted attempts stop scheduling for inspection. An active experiment cannot be overwritten by another campaign. A live H1 permits only the exact options tested; merging multiple offline winners requires its own combined live arm rather than assuming their effects add.

The gate scheduler once checked futility before acquiring a concurrency slot. Every queued coroutine passed before any results arrived, so a 100-pair panel could decide at 40 yet run all 200 games. Checking again after slot acquisition cut the serial reproduction to 80 games; concurrency four can finish at most three additional games. The first stopping decision is latched. Later rows are kept as `post_decision` and excluded from the result, including after resume. This saves compute while preserving the original scientific decision.

Implementation: `scripts/live_deploy.py`, `scripts/auto_promote.py`, `scripts/campaign_service.py`, `search/campaign.py`, `search/value_gate.py`, `scripts/ab_watch.py`, `scripts/team_ladder.py`; `docs/m5-pipeline-review.md`.

18 Limitations and research directions

The project now has a controlled live gain for a team change, in addition to its early offline search result. It does not have a proven process for continually improving turn decisions, competitive-level play, or an end-to-end optimal policy. The unsuccessful campaigns constrain the next questions more usefully than another list of promising offline metrics.

1. **Transfer from a surrogate remains the main research gap.** Better outcome log loss, reply accuracy, probe retention and sampled-game security each failed to predict stronger play in specific campaigns. C12's small duality gaps certify its finite matrices, not correct leaves, beliefs, missing replies or information-consistent continuations. Future value work should distinguish prediction from counterfactual action ranking and preserve information constraints across hidden worlds [1].

2. **C13 identifies a team–pilot interaction.** Its +61.9-Elo estimate concerns two exact teams under C5 search and C4 preview on this ladder. It is not an estimate of human tournament

team quality, an improvement in learned turn weights, or a decomposition into individual species effects. Optional stopping and selecting challengers can inflate the winning estimate. The subsequent 1344 ± 33 cohort is useful follow-up evidence but not another randomized comparison.

3. **C14 remains an experiment.** The bot rarely using Venusaur is a diagnostic, and human usage is an observational lower bound on being brought. Making the bot resemble humans does not guarantee more wins: C9 already changed Protect frequencies without demonstrating an advantage. Retain the fixed stopping rule instead of promoting from appealing plans or a short rating streak.

4. **Sequential uncertainty needs stronger calibration.** The normal-approximation live test has nominal .05/.10 parameters but is not an exact sequential likelihood-ratio test. Repeated opponents, rating feedback, factorial interactions, variance estimation and several successive campaigns complicate error rates. Ordinary performance intervals do not account for repeated looks. A future protocol could preregister block-aware inference or anytime-valid confidence sequences and a family-wise error budget; those methods are not implemented or retrospectively credited here.

5. **Missingness and treatment delivery need continued auditing.** Unfinished rooms are excluded until a terminal arrives, while forfeit wins count. Outages could still differ by arm. Compare assignment-to-terminal rates, baseline fallbacks and actual team/preview delivery in addition to completed outcomes. A source fix, browser repair or model change during a segment should not silently become a strategic gain.

6. **Beliefs and state remain approximate.** Usage marginals omit correlations among moves, items, spreads and teammates. Speed constraints and public facts help but do not make a calibrated posterior. Mega forcing, omitted opponent PP, incomplete history restoration, unseeded duration draws and determinized continuation decisions remain limitations. Correlated-set beliefs and information-consistent continuation policies are research candidates requiring fresh tests, not established fixes.

7. **Compute is a means, not a result.** The M5 accelerates fitting, but CPU simulator evaluation dominates campaign time. C6 did not validate spending more compute on the same estimator, and C10's screening efficiency did not make its combined endgame candidate win. Further hardware optimization should preserve the evaluated policy and report simulation cost, latency, legality and strength separately.

The software's strongest achievement is an auditable loop from a recorded failure to a fixed candidate, frozen inputs, persistent evaluation and an explicit deployment decision. Its scientific achievement at this cutoff is narrower and concrete: a team change beat the prior team in a controlled live comparison after many plausible search changes did not. The remaining task is to turn more accurate models and better search objectives into similarly measured improvements, without rewriting negative results as successes.

A Notation

Symbol	Meaning
s_t, h_t	Latent simulator state; information available before decision t.

Symbol	Meaning
r, $\mathcal{A}(r)$	A Showdown request and its request-visible candidate choices.
x_t, z_a	State vector in $\mathbb{R}^{384}$; action vector in $\mathbb{R}^{192}$.
q_0, f_θ, ℓ_θ	Prior (last action coordinate), learned residual, full logit.
τ	Policy temperature; 0.25 in the historical actor checkpoint; current preview models are separate.
π_θ, V_θ	Tempered masked policy; critic.
$s(a)$, κ, g	Strategic turn score, pessimism weight, squash $4s/(4+\lvert s\rvert)$.
Y, T, G_t	Terminal outcome, recorded decision count, discounted return.
γ, λ	Discount 0.99 and GAE parameter 0.95.
$\widehat{A}_t$, $\widetilde{A}_t$, $\widehat{R}_t$	GAE estimate, standardized advantage, critic target.
ρ_t, ϵ, ω_t	Importance ratio, clip width 0.2, phase weight.
$\widehat{D_{\mathrm{KL}}}$	Per-epoch estimate $(\rho-1)-\log\rho$; stop threshold 0.03.
W, d	Admitted simulator workers; GPU duty fraction.
g, ℓ, d, p	Paired gains, losses, discordant total, exact sign-test p-value (Section 10; d also denotes duty in Section 9).
n, m	Games per policy and promotion margin.
w, K, M_w	Sampled world, initial world count (6 live, up to 12 adaptively), per-world payoff matrix.
$\bar{x}_w$, $\bar{y}_w$, q_w	Regret-matching averages; blended opponent model.
$U_w(a)$, a^*	Value of our action in world w; chosen action.
E, K_w, K_ℓ	Elo expected score; winner and loser K factors.
ψ, q_ψ	Scout parameters and executed-move distribution.
v, α_v, β_v, μ_v, σ_v	Team version and its Beta posterior summaries.

B Corrections to the first edition

First edition	Second-edition correction (9 October)
Snapshot `151393d`	A private-history hash absent from the public repository. The second edition cited public `0c00ec2`; the third uses `bee08dc`.
The actor chooses every live decision	Search chooses turns; the actor chooses previews greedily (Section 11).
Policy softmax(ℓ); example $(0.594, 0.343, 0.063)$	The policy is softmax(ℓ/τ) with $\tau = 0.25$. The deployed probabilities for the same logits are $(0.900, 0.100, 0.0001)$ (Example 6.2).
Prior: power×accuracy×STAB×type, zero at preview	This is the legacy prior. The deployed prior is the squashed scenario score, and it is nonzero at preview (Section 5.3).
Preview ordering always matters	Under the strategic profiles the bench is canonicalized: 360 candidates form 180 plans (Proposition 4.2).
"Not a hypothesis test"; gate $\max(1, \lfloor nm + 0.999 \rfloor)$	The gate is $\lceil nm \rceil$ plus an exact paired sign test at $p \le 0.05$ (Equation (27)).
The worker promotes, with a 30-minute window	The worker only stages. Promotion happens before matchmaking, with no pending ladder recording, a matching parent, hash and source generation.
Auxiliary inputs are not frozen	Evaluation freezes content-addressed input snapshots.
No mid-game reconstruction after a crash	Pending-episode resume and same-rqid resubmission exist (Section 12).

First edition	Second-edition correction (9 October)
Loss: clipped PPO + value + entropy	Now with phase weights, configurable coefficients, collecting-likelihood exclusion, quality exclusions and an epoch-level KL stop (Section 8).
Consumed "once produced and evaluated"	Consumed when the candidate is saved, before evaluation.
Opponents: heuristic plus some random; optional self	The `ladder-v1` cycle (2/8 self, 3/8 tactical, 2/8 heuristic, 1/8 random); 8% open sheets; 64-bit seeds.
Resource rule: load, memory, disk	Adds memory pressure (critical only), the swap-out rate, the storage budget, and the split between evaluator and collector.
Thompson: "probability that v has the largest *sampled* parameter"	Circular; corrected to the largest *true* parameter (Proposition 13.5).
Hashing into d coordinates	Into $d - 16$ hashed buckets; 16 are dense (Proposition 5.1).
Training mask uses a finite minimum	Divided by $\tau < 1$, the minimum overflows to $-\infty$; this is harmless (Section 6).
"Not a tree search or equilibrium solver"	True of the network. False of the deployed system, which solves per-world matrix games.
Evidence: two early reports	Section 14: more than thirty gates, the search evaluation and the live record.

C Status of second-edition defects and subsequent repairs

The 9 October review exposed several defects without fixing runtime code. Later development repaired some and left others as known limits. This article update itself changes documentation and evidence tooling only.

Finding	Status at the 31 October implementation cutoff
Both-Mega enumeration discarded double attacks	Repaired in `b12314c`: enumerate plain and Mega variants, then require a legal single activation under the default policy. The old `megaFix` helper is still a no-op; the repair is in `sideChoices`.
Mega always forced	Still a baseline policy restriction. C7's opt-in species avoidance was not promoted. This is distinct from legal enumeration.
Own-set forme lookup	Mega/base aliases and regional sheet spellings are handled by `_our_set`; Floette-Eternal also gets the correct Mega prior. This is not a claim that every possible identity case is solved.
Unseeded durations and independent cell seeds	CRN now matches matrix streams across our rows. Ordinary reconstruction still samples some sleep/confusion counters with `Math.random`. C12 completion reuses cached serialized roots instead of resampling them.
Legacy actor HP parsing	The historical `strategic-v1` state encoder retains its legacy parser. Current search uses its own tracker and private request reconstruction; the actor defect is not evidence of incorrect HP in every search world.
Floating-point promotion margin	The historical PPO gate's $\lceil nm \rceil$ edge case remains an implementation limit, not the criterion used by the current live sequential experiments.
Dead burn constant	Still overridden by the attack-based penalty.

Finding	Status at the 31 October implementation cutoff
Incomplete world history	General last-move restoration and opponent PP remain incomplete. Choice-lock and some other effects use tracked last moves, so "no history is used" would overstate the defect. Recharge and Illusion have explicit later repairs.
Whole-row/column invalidation	The ordinary matrix path still removes a row/column after rejected simulator choices. C12 additionally demands complete comparable rows, but is disabled after its failed strength gate.
Cross-room answered-request collision	Repaired in `4b2800f`; a major confound in attributing the C5 deployment segment to search alone.
Illusion, Recharge, failed Protect and Ally Switch	Explicit tracker and engine repairs, covered by recorded-position and simulator regressions. See Section 17; no isolated Elo effect is claimed.
Hung browser page bypassed its own stall check	Independent watchdog thread added in `f22fa2c`; exit/restart recovery after a nonresponsive page, with the current room resumed.
Futility did not stop queued games	Recheck after simulator-slot acquisition, immutable first decision, and explicit post-decision rows. Resume preserves the same decision.

D Changes from the second edition

The third edition supersedes four potentially misleading readings of the 9 October snapshot. The actor is retained as historical machinery rather than the current baseline preview. Several defects listed as open are now repaired, but residual determinization and state limitations remain. The 41-game rating climb is preserved as a preliminary observation rather than used to claim continuing progress through the plateau. Finally, candidate promotion is no longer described as universally requiring the old paired PPO test: later protocols include a documented C5 exception, pooled offline filters and direct live A/B decisions. C13 is the controlled positive result; C14 is not yet a result.

E Source-to-claim map

Source	Claim or object
`ps_mcp_server.py`, `ps_client.py`, `harness.py`	41 MCP tools; login and transport; team CRUD; legacy loop.
`battle_state.py`	`observe`; `legal_choices`; joint predicates; HP parsers.
`ml/features.py`	Dimensions 384/192; signed hashing; profiles; bench canonicalization; prior placement and squash.
`ml/strategy.py`, `ml/tactics.py`, `ml/opening.py`	Scenario hypotheses; deterministic resolution; risk aggregate; damage; preview score.
`ml/model.py`	97,826-parameter network; temperature; GAE; likelihood validation; PPO, BC and KL stop; duty cycle.
`ml/brain.py`, `ml/recording.py`, `ml/postgame.py`	Decisions, steps, snapshots, terminal records, resume.
`ml/data_quality.py`	Trajectory exclusion rules.

Source	Claim or object
`ml/storage.py`, `ml/input_snapshot.py`, `ml/data_budget.py`	Schema, fingerprints, collected checkpoints, frozen inputs, budget.
`ml/pipeline.py`, `ml/worker.py`	Supervisor; lanes; collection curriculum; learner; evaluator.
`ml/resources.py`, `ml/batching.py`, `ml/reload.py`	Admission and guards; MPS backend; tensor cache; source generation.
`ml/promotion.py`	Paired gate; sign test; staging; promotion.
`ml/scout.py`, `ml/teams.py`, `ml/research.py`	Scout; Beta/Thompson team choice; research prior.
`search/agent.py`, `search/engine.cjs`, `search/priors.py`, `search/tracker.py`	Worlds; enumeration; screening; regret matching; blend; decision; evaluation.
`ml/browser.py`, `scripts/browser_play.py`, `scripts/live_watch.py`	Browser turn; guards; recovery; viewer.
`simulator.cjs`, `ml/simulator.py`	Official simulation and validation; player streams; local opponents.
`experiments/`, `notebooks/`	Offline PPO matrix and analysis.
`search/human/`	Public-replay extraction; imitation, preview and team-effect models.
`search/config.py`, `search/speed.py`, `search/belief.cjs`	C5 baseline, speed constraints, opt-in C12 sampled-game solver.
`search/ab_test.py`, `search/elo_monitor.py`	Balanced factorial assignment, terminal joins, approximate sequential test, performance rating.
`search/campaign.py`, `scripts/team_ladder.py`	Persistent frozen offline panels and king-of-the-hill live comparisons.
`scripts/live_deploy.py`, `scripts/auto_promote.py`	Tested pinned source deployment and graceful restart.
`collect_evidence.py`, `evidence-20261031.json`	Read-only public aggregate audit and numerical verification for this edition.

F Reproduction and build

Build from `docs/paper/` with `tectonic main.tex` (or `latexmk -pdf main.tex`). The figures are TikZ and pgfplots, with no external images. To reproduce an experiment, record at least:

1. repository and simulator revisions, format, dependency versions, feature profile, temperatures, device and seeds;
2. exact teams with their order and generated-spread flags, learner side, opponent policy and revision;
3. frozen input snapshots, collection mode and terminal, unfinished and rejected counts;
4. for search: worlds, engines, screening sizes, the usage month and cutoff, α, τ_o, iterations and the seed;
5. the evaluation unit (game, seed or opponent team), the panel's independence from development, and the multiplicity plan.

Use an isolated data root for experiments, because the ordinary root is live learning state. The offline test suite checks the implemented contracts (enumeration, masking, collection modes, likelihoods, terminal recording, reloads, scout non-anticipation, promotion predicates). Passing it is not a measurement of playing strength.

F.1 Reproducing the third-edition audit

The tracked aggregate snapshot is fixed; a document build never refreshes it. Run `python3 collect_evidence.py -verify` in `docs/paper/` to recompute offline exact tails, both C13 round scores, Elo transforms and normal-approximation LLRs without private inputs. The snapshot records the source commit, UTC cutoff, manifest checksums, first-stopping campaign receipts, performance cohorts and C14's unresolved status.

With the private ledgers available, `-collect -output PATH` creates a *new* snapshot by calling the repository's read-only ledger parsers. It filters completed events at the collection cutoff, deduplicates rooms, joins assignments, and truncates C13 rounds at their saved stopping counts. It publishes aggregates only. It performs no login, battle, fit, service change or promotion. Exact historical recollection also requires the underlying append-only logs and campaign receipts; the public aggregate alone cannot re-audit every original room or missing assignment.

New empirical claims are traced to public campaign documents and the frozen aggregate audit. The original bibliography remains for historical results; new game-theoretic context is verified against primary papers on robust responses and imperfect-information depth limits [1, 8].

References

[1] Noam Brown, Tuomas Sandholm, and Brandon Amos. Depth-limited solving for imperfect-information games. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://noambrown.github.io/papers/18-NIPS-Depth.pdf. Primary paper verified 2026-10-31; information-consistent continuation limits.

[2] Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):120–143, 2012. doi: 10.1109/TCIAIG.2012.2200894. URL https://eprints.whiterose.ac.uk/75048/.

[3] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishing, New York, 1978.

[4] Ian Frank and David Basin. Search in games with incomplete information: A case study using bridge card play. *Artificial Intelligence*, 100(1–2):87–123, 1998. doi: 10.1016/S0004-3702(97)00082-9. URL https://www.dai.ed.ac.uk/daidb/papers/documents/rp780.html.

[5] Sergiu Hart and Andreu Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica*, 68(5):1127–1150, 2000. doi: 10.1111/1468-0262.00153. URL https://doi.org/10.1111/1468-0262.00153.

[6] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830. URL https://doi.org/10.1080/01621459.1963.10500830.

[7] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL https://www.jstor.org/stable/4615733.

[8] Michael Johanson, Martin Zinkevich, and Michael Bowling. Computing robust counter-strategies. In *Advances in Neural Information Processing Systems*, volume 20, 2007. URL https://poker.cs.ualberta.ca/publications/NIPS07-rnash.pdf. Primary paper verified 2026-10-31; motivates the robustness tradeoff, not the project's measured Elo effect.

[9] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X. URL https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf.

[10] Jeffrey Long, Nathan R. Sturtevant, Michael Buro, and Timothy Furtak. Understanding the success of perfect information Monte Carlo sampling in game tree search. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, pages 134–140, 2010. URL https://ojs.aaai.org/index.php/AAAI/article/view/7562.

[11] Richard D. McKelvey and Thomas R. Palfrey. Quantal response equilibria for normal form games. *Games and Economic Behavior*, 10(1):6–38, 1995. doi: 10.1006/game.1995.1023. URL https://resolver.caltech.edu/CaltechAUTHORS:20171128-164515991.

[12] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996. URL https://link.springer.com/article/10.1007/BF02295996.

[13] Microsoft. Playwright, n.d. URL https://playwright.dev/. Browser automation library; accessed 2026-10-09.

[14] Model Context Protocol contributors. Model context protocol specification: Tools, 2025. URL https://modelcontextprotocol.io/specification/2025-06-18/server/tools. Specification version 2025-06-18; accessed 2026-10-07.

[15] Model Context Protocol contributors. Model context protocol specification: Transports, 2025. URL https://modelcontextprotocol.io/specification/2025-06-18/basic/transports. Specification version 2025-06-18; accessed 2026-10-07.

[16] OpenClaw contributors. Manage saved mcp servers, n.d. URL https://docs.openclaw.ai/cli/mcp/registry. Accessed 2026-10-07.

[17] Pokémon Showdown. How the ladder works, n.d. URL https://pokemonshowdown.com/pages/ladderhelp. Accessed 2026-10-09.

[18] Pokémon Showdown contributors. Battlestream and player stream implementation, n.d.. URL https://github.com/smogon/pokemon-showdown/blob/c046106cbe075931b1ff8d8b800ff5be47a85f96/sim/battle-stream.ts. Pinned official simulator source; inspected through repository dependency.

[19] Pokémon Showdown contributors. Pokémon showdown client–server protocol, n.d.. URL https://github.com/smogon/pokemon-showdown/blob/c046106cbe075931b1ff8d8b800ff5be47a85f96/PROTOCOL.md. Pinned simulator source; inspected through repository dependency.

[20] Pokémon Showdown contributors. Pokémon showdown: Pinned simulator and format source, n.d.. URL https://github.com/smogon/pokemon-showdown/tree/c046106cbe075931b1ff8d8b800ff5be47a85f96. Commit c046106cbe075931b1ff8d8b800ff5be47a85f96; project dependency pin.

[21] Pokémon Showdown contributors. Pokémon showdown simulator protocol, n.d.. URL https://github.com/smogon/pokemon-showdown/blob/master/sim/SIM-PROTOCOL.md. Accessed 2026-10-07.

[22] PyTorch contributors. torch.mps.set_per_process_memory_fraction, n.d. URL https://pytorch.org/docs/stable/generated/torch.mps.set_per_process_memory_fraction. Accessed 2026-10-09.

[23] John Schulman. Approximating KL divergence, 2020. URL http://joschu.net/blog/kl-approx.html. Blog post, 7 March 2020.

[24] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, 2015. URL https://proceedings.mlr.press/v37/schulman15.html.

[25] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016. URL https://arxiv.org/abs/1506.02438.

[26] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347. URL https://arxiv.org/abs/1707.06347.

[27] Richard D. Smallwood and Edward J. Sondik. The optimal control of partially observable markov processes over a finite horizon. *Operations Research*, 21(5):1071–1088, 1973. doi: 10.1287/opre.21.5.1071. URL https://pubsonline.informs.org/doi/10.1287/opre.21.5.1071.

[28] Smogon University. Pokémon showdown usage statistics (chaos JSON), september 2026, 2026. URL https://www.smogon.com/stats/. Format gen9championsvgc2026regmc, cutoff 0; 1,631,943 battles.

[29] Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html.

[30] The Pokémon Company. Regulation set m-c (updated on september 9), 2026. URL https://champions-news.pokemon-home.com/en/page/816.html. Accessed 2026-10-07.

[31] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933. doi: 10.1093/biomet/25.3-4.285. URL https://academic.oup.com/biomet/article-abstract/25/3-4/285/200862.

[32] Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16 (2):117–186, 1945. doi: 10.1214/aoms/1177731118. URL https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-16/issue-2/Sequential-Tests-of-Statistical-Hypotheses/10.1214/aoms/1177731118.full. Classical sequential-test context; the project's estimated-variance normal approximation is qualified separately.

[33] Kilian Weinberger, Anirban Dasgupta, Josh Attenberg, John Langford, and Alex Smola. Feature hashing for large scale multitask learning. *arXiv preprint arXiv:0902.2206*, 2009. URL https://arxiv.org/abs/0902.2206. Version 5 revised 2010.

[34] WHATWG. HTML living standard: Server-sent events, n.d. URL https://html.spec.whatwg.org/multipage/server-sent-events.html. Accessed 2026-10-09.

[35] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2007. URL https://proceedings.neurips.cc/paper/2007/hash/08d98638c6fcd194a4b1e6992063e944-Abstract.html.